

CS

A Compact-Simple Bimodal Lattice Signature

Shuaigang Li^{1,2}, Liang Bai^{1,2}, Zhongxiang Zheng³, Keteng Huang^{1,2}, Yan Ding^{1,2},
Yongbiao Ma¹, Yuchen Wang¹

¹Xingtang Telecommunications Technology Co., Ltd.

²Beijing Key Laboratory of Post-Quantum Cryptography

³Communication University of China

June 25, 2026

Contents

1	Introduction	2
1.1	Design Rationale	2
1.2	Advantages and Disadvantages	4
1.2.1	Advantages	5
1.2.2	Disadvantages	6
2	Preliminaries	6
2.1	Notation and Distributions	6
2.2	NTT Representation	7
2.3	Generation of Random Ring Elements	8
2.4	Challenge Polynomial Generation	8
2.5	High/Low Bits and Hints	8
2.6	rANS Encoding	10
2.7	<i>Reject</i> Algorithm	10
3	Signature Scheme	13
3.1	Rejection Sampling Algorithm	14
3.2	Simplified CS Signature Scheme	16
3.3	Full CS Signature Scheme	17
3.4	Correctness and Signature Repetition Count	20
3.5	Parameter Sets and Selection	22
4	Security Analysis	24
4.1	Cryptographic Definitions	24
4.2	Signature Scheme Security	27
4.3	Concrete Security Strength	30
4.4	Other Security Features	31
5	Performance and Features	31
5.1	Performance	31
5.2	Features	33
	References	34
	Appendix	36
	Appendix A Properties of the centered multinomial distribution	36
	Appendix B Lemmas for the Rejection Sampling Algorithm	37

1 Introduction

Depending on how the signature is generated, Fiat-Shamir (FS) lattice-based signature schemes can be divided into two categories: unimodal FS schemes and bimodal FS schemes. Unimodal FS schemes (such as the NIST-standardised ML-DSA scheme [LDK⁺22, FIPS204]) are easy to implement without any floating-point operations throughout the whole process and have good side-channel resistance, but their public-key and signature sizes are large. Bimodal FS schemes (such as the Korean-standardised HAETAE scheme [CHE⁺24, CHE⁺23]) achieve more compact public-key and signature sizes, but often involve floating-point operations and are more vulnerable to side-channel attacks.

To break through the bottlenecks of both approaches simultaneously, we designed the signature scheme CS. *C* is the initial of Cun (“寸”), and also the initial of Compact, representing its compactness in public-key and signature sizes. *S* is the initial of Suan (“算”), and also the initial of Simple, representing that it relies entirely on simple integer operations. The CS scheme is free of floating-point operations throughout the whole process, while possessing both side-channel resistance and compactness.

CS is a bimodal FS lattice-based signature scheme based on the Module Learning With Errors (MLWE) problem and the Module Short Integer Solution (MSIS) problem over module lattices. The scheme uses a simple and easy-to-sample bimodal Centered Multinomial Distribution as the mask distribution, and avoids floating-point operations in the rejection sampling algorithm through integer operations, thereby possessing good side-channel resistance. CS also supports efficient public-key compression techniques and uses a larger challenge-polynomial space, thus achieving compact public-key and signature sizes. In addition, CS proposes a more flexible iterative bimodal design to improve the scheme’s efficiency. Overall,

$$\text{CS} = \text{module lattices} + \text{centered multinomial distribution} + \text{efficient public-key compression} + \\ \text{flexible iterative bimodal} \Rightarrow \text{compact sizes} + \text{no floating-point throughout}$$

1.1 Design Rationale

FS lattice-based signature schemes [Lyu09, Lyu12, BG14] are variants of Schnorr’s signature scheme [Sch91]; the NIST-standardised ML-DSA scheme [LDK⁺22, FIPS204] and the Korean-standardised HAETAE scheme [CHE⁺24, CHE⁺23] are both FS lattice-based signature schemes. In early FS lattice-based signature schemes, the public key usually consists of a matrix $\mathbf{A}$ and a vector $\mathbf{b}$, and the secret key is a vector $\mathbf{s}$ satisfying $\mathbf{As} = \mathbf{b} \pmod{q}$. Given a message μ , the signing algorithm first generates a mask vector $\mathbf{y}$, then computes the commitment $\mathbf{w} = \mathbf{Ay}$, the challenge polynomial $c = H(\mathbf{w}, \mu)$, and the response polynomial $\mathbf{z} = \mathbf{y} + c\mathbf{s}$, and finally outputs the signature $(\mathbf{z}, c)$ with a certain probability via the rejection sampling algorithm, or outputs $\perp$ otherwise. Given a message μ and its signature $(\mathbf{z}, c)$, the verifier computes the commitment polynomial $\mathbf{w}' = \mathbf{Az} - c\mathbf{b} = \mathbf{A}(\mathbf{y} + c\mathbf{s}) - c\mathbf{b} = \mathbf{Ay} = \mathbf{w} \pmod{q}$; the verification succeeds if $\mathbf{z}$ is small and $c = H(\mathbf{w}', \mu)$, and fails otherwise. In bimodal FS lattice-based signature schemes,

the public and secret keys usually satisfy $\mathbf{As} = q\mathbf{j} \bmod (2q)$ (where $\mathbf{j}$ is a unit vector), and the signing algorithm outputs $\mathbf{z} = \mathbf{y} + \mathbf{cs}$, $\mathbf{z} = \mathbf{y} - \mathbf{cs}$, or $\perp$, each with a certain probability.

The core features of the CS scheme we designed are that it is free of floating-point operations throughout the whole process and that the sum of its public-key and signature sizes is small. To achieve these two goals, the CS scheme uses the following key designs on the basis of traditional bimodal FS signature schemes:

Bimodal centered multinomial distribution. This is a key component for implementing CS without floating-point operations throughout the whole process. The key to the floating-point-free design lies in two points: first, the sampling process of the mask $\mathbf{y}$ is free of floating-point operations; second, the rejection probability computation in the rejection sampling algorithm is free of floating-point operations. There are several common distributions for the mask $\mathbf{y}$, including Gaussian distributions, uniform distributions over a hypersphere, and uniform distributions over a hypercube, etc. Gaussian distributions and uniform distributions over a hypersphere both involve floating-point operations during sampling. The sampling and rejection-probability computation of the uniform distribution over a hypercube do not involve floating-point operations, but this distribution is incompatible with the bimodal design, resulting in a high signature repetition count; to reduce the repetition count, its parameters must be set large, leading to large signature and public-key sizes. For this reason, we use the bimodal centered multinomial distribution. The Centered Multinomial Distribution is a natural generalisation of the centered binomial distribution; its sampling algorithm is simple and involves no floating-point operations. Moreover, the probability mass function of the Centered Multinomial Distribution can be expressed exactly as the ratio of two integers, avoiding floating-point operations, so that the rejection probability can be computed using only integer operations. Therefore, the whole CS scheme is guaranteed to be free of floating-point operations. However, this makes the rejection-probability computation involve the product of multiple integers. To avoid the inefficiency caused by large integers in multiplication, we use integer truncation to ensure efficiency, and use Rényi divergence to prove that the effect of truncation on the scheme’s security is controllable.

Efficient public-key compression. Existing bimodal FS lattice-based signature schemes are usually incompatible with efficient public-key compression mechanisms: for example, [D⁺LL13, WKH⁺25] have no public-key compression, and [CHE⁺24] compresses the public key by at most 1 bit. Like the ML-DSA scheme, the CS scheme can compress the public key efficiently, which not only reduces the public-key size but also improves the scheme’s flexibility.

Larger challenge-polynomial space. In traditional bimodal FS lattice-based signature schemes, the coefficients of the challenge polynomial are chosen with a fixed Hamming weight over $\{0, 1\}$. The CS scheme uses a fixed Hamming weight over $\{0, \pm 1\}$. With the same challenge entropy, the fixed Hamming weight of the CS challenge polynomial is smaller, which reduces the signature size to some extent.

Flexible iterative bimodal design. In 2025, [Gar25] proposed iterative bimodal FS lattice-based signature schemes. The iterative bimodal design can effectively reduce the signature repetition count of the signing algorithm, thereby improving signing efficiency. Unlike the bimodal

Gaussian design of [Gar25], the CS scheme uses the bimodal centered multinomial distribution. Another key factor affecting the efficiency of the CS signing algorithm is the complexity of the rejection-probability computation. Using the iterative bimodal design of [Gar25] can reduce the signature repetition count, but the increased number of iterations raises the complexity of the rejection-probability computation. On the other hand, using a non-iterative bimodal design [DDLL13, CHE⁺24, WKH⁺25] lowers the complexity of the rejection-probability computation but increases the signature repetition count. In order to better balance these two factors and improve the overall efficiency of the CS signing algorithm, we introduce a parameter τ' to better balance the signature repetition count and the rejection-probability computation complexity, thereby improving signing efficiency.

In summary, on the basis of traditional bimodal FS lattice-based signature schemes, we use a simple and easy-to-sample bimodal Centered Multinomial Distribution and implement the corresponding rejection sampling process through integer operations, so that the CS scheme is free of floating-point operations throughout the whole process. In addition, the CS scheme uses efficient public-key compression techniques and a larger challenge-polynomial space to reduce signature and public-key sizes. At the same time, we propose a flexible iterative bimodal design to improve the scheme's efficiency. Ultimately, we have designed the compact lattice-based signature scheme CS, which is free of floating-point operations throughout the whole process.

1.2 Advantages and Disadvantages

Table 1 shows the sizes of the HAETAE, ML-DSA, and CS schemes. Overall, the CS signature scheme has advantages in public-key and signature sizes: its public key is the smallest of the three, and the sum of its public-key and signature sizes is almost the smallest. Table 2 shows the efficiency of the three schemes; the hash function used is SM3. Overall, the CS scheme shows good efficiency in key generation and signature verification: key generation is the fastest among the three, and verification efficiency is also high.

Table 1: Size comparison of signature schemes (bytes)

algorithms	public key	signature	sum	secret key
HAETAE-120	992	1474	2466	1408
ML-DSA-44	1312	2420	3732	2560
CS-128	976	1548	2524	1888
HAETAE-260	2080	2948	5028	2752
ML-DSA-87	2592	4627	7219	4896
CS-256	1760	3164	4924	3968
CS-512	4288	5975	10263	7808

Table 2: Efficiency comparison of signature schemes (kcycles)

algorithms	KeyGen	Sign	Verify
HAETAE-120	2722	11946	456
ML-DSA-44	996	2371	930
CS-128	503	4009	536
HAETAE-260	4892	22605	1160
ML-DSA-87	2885	5079	2789
CS-256	809	13083	1028
CS-512	2466	45269	3127

1.2.1 Advantages

1. **No floating-point operations throughout the whole process, easy to protect against side-channel attacks.** Bimodal FS signature schemes such as HAETAE often involve floating-point operations, making side-channel protection difficult. Like ML-DSA, the CS scheme is free of floating-point operations throughout the whole process and involves only simple integer operations, so it is easy to protect against side-channel attacks.
2. **Small public-key and signature sizes.** As shown in Table 1, at the 128 and 256 security levels, the public-key size of the CS scheme is the smallest among the three, approximately 75% and 68% of that of ML-DSA, respectively. The signature size of the CS scheme is also almost the smallest among the three, approximately 64% and 69% of that of ML-DSA, respectively. The sum of the public-key and signature sizes of the CS scheme is also almost the smallest among the three, approximately 68% and 69% of that of ML-DSA, respectively.
3. **Fast key generation and signature verification.** As shown in Table 2, at the 128 and 256 security levels, CS key generation is approximately 5.4 and 6.0 times faster than HAETAE, and approximately 2.0 and 3.6 times faster than ML-DSA. CS signature verification is approximately 1.7 and 2.7 times faster than ML-DSA, and comparable to HAETAE. Therefore, the CS scheme has good efficiency in both key generation and signature verification.
4. **Flexible parameter selection.** Besides the flexibility of the underlying module-lattice hard problems, compared with traditional lattice-based signature schemes, we have more algorithm parameters that can be adjusted. By optimising the parameter configuration, we can better balance security, size, and efficiency to suit application scenarios with different requirements.

1.2.2 Disadvantages

The signature generation speed of the CS scheme is faster than that of HAETAE but slower than that of ML-DSA. As shown in Table 2, at the 128 and 256 security levels, the signature generation speed of CS is approximately 59% and 39% of that of ML-DSA, respectively. However, the signature generation speed of the CS scheme is still better than that of HAETAE, approximately 3.0 and 1.7 times faster, respectively.

2 Preliminaries

2.1 Notation and Distributions

This scheme uses $\lceil x \rceil$ and $\lfloor x \rfloor$ to denote the ceiling and floor of a rational number x , respectively; $\log$ denotes the logarithm to base 2; and $LSB(x)$ denotes the least significant bit of the integer x . For positive integers x and a , $x \gg a$ denotes shifting x right by a bits, $len(x)$ denotes the bit length of x , $[x] = \{0, 1, \dots, x-1\}$, and $\binom{x}{a}$ denotes the number of combinations of choosing a elements from x distinct elements. For an integer x and a positive integer q , $y = x \bmod q$ denotes the unique $y \in [q]$ such that $(y - x) \bmod q = 0$, and $y = x \bmod^{\pm} q$ denotes the unique $y \in [-q/2, q/2)$ such that $(y - x) \bmod q = 0$.

By default, this scheme uses bold lowercase letters for column vectors (e.g., $\mathbf{y}$) and bold uppercase letters for matrices (e.g., $\mathbf{A}$). We use $(\mathbf{y}^{\top}, \mathbf{z}^{\top})^{\top}$ to denote concatenating the column vector $\mathbf{z}$ below the column vector $\mathbf{y}$, and $(\mathbf{y}|\mathbf{z})$ to denote concatenating the column vector $\mathbf{z}$ to the right of the column vector $\mathbf{y}$. The latter can naturally be generalised to vectors and matrices (e.g., $(\mathbf{A}|\mathbf{B})$) and to matrices and matrices (e.g., $(\mathbf{A}|\mathbf{B})$).

$\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ denotes a polynomial ring, where n is a power of 2. We define $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$, where q and n satisfy $2n \mid (q - 1)$ so that the Number Theoretic Transform (NTT) can be used to accelerate polynomial multiplication. This scheme also defines $\mathcal{R}_{2q} = \mathbb{Z}_{2q}[x]/(x^n + 1)$. In addition, $\mathcal{R}_q^k$ and $\mathcal{R}_q^{k \times \ell}$ denote k -dimensional vectors over $\mathcal{R}_q$ and $k \times \ell$ -dimensional matrices over $\mathcal{R}_q$, respectively. Accordingly, $\mathcal{R}_{2q}$ can also be generalised to its vector and matrix forms.

An integer random variable X follows the Centered Multinomial Distribution (CMD) $\mathcal{D}_{a,b}$ with parameters (a, b) if X is generated as

$$\sum_{i=0}^{a-1} (x_i - y_i)$$

where $x_i \leftarrow [2^b]$ and $y_i \leftarrow [2^b]$. Furthermore, we define $\mathcal{D}_{a,b,r}$ as the distribution obtained by shifting $\mathcal{D}_{a,b}$ right by r units. We define $\overline{\mathcal{D}}_{a,b,r}^t$ as the new distribution formed by truncating $\mathcal{D}_{a,b,r}$ to the interval $[-t, t]$ (in this scheme $t \leq a(2^b - 1) - |r|$). The bimodal Centered Multinomial Distribution $\mathcal{D}_{a,b,\pm r}$ takes a sample from $\mathcal{D}_{a,b,r}$ with probability $\frac{1}{2}$ and a sample from $\mathcal{D}_{a,b,-r}$ with probability $\frac{1}{2}$; the corresponding $\overline{\mathcal{D}}_{a,b,\pm r}^t$ is the new distribution formed by truncating $\mathcal{D}_{a,b,\pm r}$ to $[-t, t]$. The Centered Multinomial Distribution and its related distributions can be generalised

to vectors, polynomials, and polynomial vectors. $\mathbf{x} \sim \mathcal{D}_{a,b}^k$ means that each component of the k -dimensional vector $\mathbf{x}$ is independent and $\mathbf{x}_i \sim \mathcal{D}_{a,b}$; $x \sim \mathcal{D}_{a,b}^{\mathcal{R}}$ means that the coefficients of the polynomial $x \in \mathcal{R}$ are independently drawn from $\mathcal{D}_{a,b}$; $\mathbf{x} \sim \mathcal{D}_{a,b}^{\mathcal{R}^k}$ means that each coefficient of each polynomial of the polynomial vector $\mathbf{x} \in \mathcal{R}^k$ is independently drawn from $\mathcal{D}_{a,b}$. When no confusion arises, all of the above distributions are abbreviated as $\mathcal{D}_{a,b}$. Similarly, $\mathcal{D}_{a,b,r}$, $\overline{\mathcal{D}}_{a,b,r}^t$, $\mathcal{D}_{a,b,\pm r}$, and $\overline{\mathcal{D}}_{a,b,\pm r}^t$ can also be generalised to vectors, polynomials, and polynomial vectors. In addition, $\mathcal{S}_\eta$ denotes the uniform integer distribution over $[-\eta, \eta]$, and $\mathcal{S}_1$ is abbreviated as $\mathcal{S}$; $\mathcal{S}_\eta$ can also be generalised to vectors, polynomials, and polynomial vectors.

For a distribution $\mathcal{D}$, $x \leftarrow \mathcal{D}$ means that x is chosen uniformly at random from $\mathcal{D}$; a polynomial $x \leftarrow \mathcal{D}$ means that the coefficients of the polynomial $x \in \mathcal{R}$ are chosen at random from $\mathcal{D}$; a polynomial vector $\mathbf{x} \leftarrow \mathcal{D}^k$ means that each coefficient of each polynomial component $\mathbf{x}_i$ of $\mathbf{x}$ is chosen at random from $\mathcal{D}$. In addition, $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}$ means that the $k \times \ell$ polynomial matrix $\mathbf{A}$ is chosen uniformly at random from $\mathcal{R}_q^{k \times \ell}$.

2.2 NTT Representation

The polynomial rings involved in the CS signature scheme are $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$ and $\mathcal{R}_{2q} = \mathbb{Z}_{2q}[x]/(x^n + 1)$, where n is a power of 2 and q is a prime satisfying $q \equiv 1 \pmod{2n}$. The multiplicative group $\mathbb{Z}_q^*$ contains a primitive $2n$ -th root of unity. However, the modulus $2q$ is composite, so performing the Number Theoretic Transform (NTT) directly modulo $2q$ is not feasible. For this reason, the computation is first performed modulo q using the NTT, and then the result is lifted to $\mathbb{Z}_{2q}$ via the Chinese Remainder Theorem (CRT).

Since $q \equiv 1 \pmod{2n}$, the multiplicative group $\mathbb{Z}_q^*$ contains a primitive $2n$ -th root of unity ζ . In order to optimise the underlying arithmetic structure, we choose not to decompose the polynomial completely into linear factors, but to decompose it into the product of $n/2$ quadratic factors. For $i = 0, 1, \dots, n/2 - 1$, we pair the conjugate roots ζ^{2i+1} and $\zeta^{-(2i+1)}$, so that the cyclotomic polynomial $x^n + 1$ factors over $\mathbb{Z}_q$ as

$$x^n + 1 = \prod_{i=0}^{n/2-1} (x - \zeta^{2i+1})(x - \zeta^{-(2i+1)}) = \prod_{i=0}^{n/2-1} (x^2 - \eta_i x + 1),$$

where $\eta_i = \zeta^{2i+1} + \zeta^{-(2i+1)} \pmod{q}$.

By the Chinese Remainder Theorem (CRT), the ring $\mathcal{R}_q$ is isomorphic to the direct product of $n/2$ quadratic polynomial quotient rings:

$$\mathcal{R}_q \cong \prod_{i=0}^{n/2-1} \mathbb{Z}_q[x]/(x^2 - \eta_i x + 1).$$

In this representation, a polynomial $a \in \mathcal{R}_q$ is mapped isomorphically to $n/2$ linear polynomials. Specifically, denote its NTT representation by the vector of these $n/2$ remainders:

$$\hat{a} = \text{NTT}(a) = (\hat{a}_0(x), \hat{a}_1(x), \dots, \hat{a}_{n/2-1}(x)),$$

where each component $\hat{a}_i(x) = a(x) \bmod (x^2 - \eta_i x + 1)$ has the form $\hat{a}_i(x) = c_{i,0} + c_{i,1}x$.

In this frequency-domain representation, the multiplication of two polynomials $a, b \in \mathcal{R}_q$ is transformed into component-wise polynomial multiplication modulo $x^2 - \eta_i x + 1$ on the corresponding $n/2$ components:

$$a \cdot b = \text{NTT}^{-1}(\text{NTT}(a) \circ \text{NTT}(b)),$$

where $\circ$ denotes the basecase multiplication of the remainder components $\hat{a}_i(x)$ and $\hat{b}_i(x)$ modulo $x^2 - \eta_i x + 1$, and NTT^{-1} is the inverse NTT.

2.3 Generation of Random Ring Elements

In the CS scheme, the public-key matrix, secret-key vectors, and signature mask are all generated deterministically from short seeds by a hash function. The three expansion functions are briefly described below.

- *ExpandA*(ρ_0): takes as input a short seed ρ_0 and outputs the NTT form of the public-key matrix $\mathbf{A}' \in \mathcal{R}_q^{k \times \ell}$. It derives a child seed for each entry of the matrix from ρ_0 as the root seed, and then generates uniformly random polynomials over $\mathcal{R}_q$ by rejection sampling, so that the whole matrix can be reconstructed reproducibly.
- *ExpandS*(ρ_1): takes as input a short seed ρ_1 and outputs the secret vectors $\mathbf{s}_1 \in \mathcal{S}^\ell$ and $\mathbf{s}_2 \in \mathcal{S}^k$. It derives child seeds for each component of the two polynomial vectors from ρ_1 as the root seed, and then samples polynomial vectors whose coefficients lie in $\mathcal{S}$.
- *ExpandMask*(ρ', κ, a, b): takes as input the signing-session seed ρ' , a retry counter κ , and the centered-polynomial-distribution parameters (a, b) , and outputs a mask polynomial vector following $\mathcal{D}_{a,b}$. It derives a child seed for each component from ρ' , combined with the counter κ , and then samples according to the centered multinomial distribution. The counter κ is incremented on each signing retry, so that different retries use different masks.

2.4 Challenge Polynomial Generation

The challenge polynomial c is generated by the same sampling method as the ML-DSA scheme; see [Algorithm 1](#).

2.5 High/Low Bits and Hints

Definition 2.1 (High/low bits functions). *Let $r \in \mathbb{Z}$ and let α be a positive even integer. The high and low bits of r are defined as*

$$\text{HighBits}(r, \alpha) = r_1, \quad \text{LowBits}(r, \alpha) = r_0,$$

where $r_1 = \lfloor \frac{r + \frac{\alpha}{2}}{\alpha} \rfloor$ and $r_0 = r \bmod \pm \alpha$.

Algorithm 1 Challenge sampling algorithm $SampleInBall(B, \tau)$

```

1:  $c_0 c_1 \cdots c_{n-1} = 00 \cdots 0$ 
2: for  $i = n - \tau$  to  $n - 1$  do
3:    $j \leftarrow \{0, 1, \dots, i\}$   $\triangleright$  rejection sampling from random bytes  $B$ 
4:    $s \leftarrow \{0, 1\}$ 
5:    $c_i = c_j$ 
6:    $c_j = (-1)^s$ 
7: end for
8: return  $c$ 

```

When $\mathbf{r}$ is a polynomial vector, $HighBits(\mathbf{r}, \alpha)$ denotes applying $HighBits$ to every coefficient of every polynomial in the vector; $LowBits(\mathbf{r}, \alpha)$ is defined analogously. The high/low bits functions satisfy the following lemma.

Lemma 2.1. *Let $\alpha > 2$ be a positive even integer and let q be a prime satisfying $\alpha \mid 2(q-1)$. For an integer $r \in \mathbb{Z}$, write $r_1 = HighBits(r, \alpha)$ and $r_0 = LowBits(r, \alpha)$. Then the following properties hold:*

1. $r_0 \in [-\frac{\alpha}{2}, \frac{\alpha}{2})$
2. $r = r_0 + \alpha \cdot r_1$
3. $r \in [0, 2q-1] \Rightarrow r_1 \in [0, \frac{2q-2}{\alpha}]$

Proof. By the definition of $\bmod^\pm$ in Section 2, property 1 holds. Since there exists an integer p such that $r = r_0 + p\alpha$ and $r_0 \in [-\frac{\alpha}{2}, \frac{\alpha}{2})$, we have

$$r_1 = \lfloor \frac{r + \alpha/2}{\alpha} \rfloor = p + \lfloor \frac{r_0 + \alpha/2}{\alpha} \rfloor = p,$$

which gives property 2, i.e. $r = r_0 + \alpha \cdot r_1$. Because $\alpha \mid 2(q-1)$, when $r = 2q-1$ we have $r_1 = \lfloor \frac{2q-1+\alpha/2}{\alpha} \rfloor = \frac{2(q-1)}{\alpha} + \lfloor \frac{\alpha/2+1}{\alpha} \rfloor = \frac{2q-2}{\alpha}$; when $r = 0$ we have $r_1 = \lfloor \frac{r+\alpha/2}{\alpha} \rfloor = 0$. Since r_1 is a non-decreasing function of r , we obtain $r_1 \in [0, \frac{2q-2}{\alpha}]$, i.e. property 3 holds. $\square$

In the signing algorithm, the hint vector $\mathbf{h}$ is computed using only the high bits. To further reduce the entropy of the high-bit values, we make a small adjustment to the above high/low bits functions, obtaining the hint high/low bits functions; see Definition 2.2.

Definition 2.2 (Hint high/low bits functions). *Let $\alpha > 2$ be a positive even integer and let q be a prime satisfying $\alpha \mid 2(q-1)$. Let $p = \frac{2(q-1)}{\alpha}$, $r \in \mathbb{Z}$, $r_1 = HighBits(r \bmod 2q, \alpha)$, and $r_0 = LowBits(r \bmod 2q, \alpha)$. The hint high and low bits of r are defined as*

$$HighBits^h(r, \alpha) = r'_1, \quad LowBits^h(r, \alpha) = r'_0,$$

where, if $r_1 = p$, then $r'_1 = 0$ and $r'_0 = r_0 - 2$; otherwise $r'_1 = r_1$ and $r'_0 = r_0$.

When $\mathbf{r}$ is a polynomial vector, $HighBits^h(\mathbf{r}, \alpha)$ denotes applying $HighBits^h$ to every coefficient of every polynomial in the vector; $LowBits^h(\mathbf{r}, \alpha)$ is defined analogously. The hint high/low bits functions satisfy the following lemma.

Lemma 2.2. *Let $\alpha > 2$ be a positive even integer and let q be a prime satisfying $\alpha \mid 2(q-1)$. Let $p = \frac{2(q-1)}{\alpha}$. For an integer $r \in \mathbb{Z}$, write $r'_1 = HighBits^h(r \bmod 2q, \alpha)$ and $r'_0 = LowBits^h(r \bmod 2q, \alpha)$. Then the following properties hold:*

1. $r'_0 \in [-\frac{\alpha}{2} - 2, \frac{\alpha}{2})$
2. $r = \alpha \cdot r'_1 + r'_0 \bmod 2q$
3. $r'_1 \in [0, p-1]$

Proof. By Lemma 2.1, $r_0 \in [-\frac{\alpha}{2}, \frac{\alpha}{2})$, so property 1 holds. By Lemma 2.1, $r = \alpha \cdot r_1 + r_0 \bmod 2q$. If $r_1 \neq p$, then $r = \alpha \cdot r'_1 + r'_0 \bmod 2q$. If $r_1 = p$, then $r = \alpha \cdot r_1 + r_0 = 2(q-1) + r'_0 + 2 = r'_0 \bmod 2q$, so property 2 holds. If $r_1 = p$, then $r'_1 = 0 \in [0, p-1]$; otherwise $r'_1 = r_1 < p$, so property 3 holds. $\square$

2.6 rANS Encoding

The CS signature scheme uses the range Asymmetric Numeral System (rANS) [Dud09, Gie14] to compress $\mathbf{z}_0$ and $\mathbf{z}_1$ in the signature, so as to make the signature size smaller.

Definition 2.3 (Range Asymmetric Numeral System (rANS) compression parameters). *Let $t > 0$ and $S \subseteq [0, 2^t - 1]$. Let $g : [0, 2^t - 1] \rightarrow \mathbb{Z} \cap (0, 2^t]$ satisfy $\sum_{x \in S} g(x) = 2^t$ and $g(x) = 0$ for all $x \notin S$. We define the following:*

- $CDF : S \rightarrow \mathbb{Z}$, defined by $CDF(s) = \sum_{y=0}^{s-1} g(y)$.
- $D^{-1} : \mathbb{Z} \rightarrow S$, where $D^{-1}(y)$ is the unique $s \in S$ satisfying $CDF(s) \leq y < CDF(s+1)$.
- $C : \mathbb{Z} \times S \rightarrow \mathbb{Z}$, defined by

$$C(x, s) = \left\lfloor \frac{x}{g(s)} \right\rfloor \cdot 2^t + (x \bmod g(s)) + CDF(s). \quad (1)$$

We call $\text{params} = (t, S, g, CDF)$ the rANS compression parameters. The rANS encoder and decoder use the same pre-agreed compression parameters; the encoder maps a sequence of symbols to an integer, and the decoder recovers the original sequence from that integer. See Algorithms 2 and 3.

2.7 Reject Algorithm

The *Reject* algorithm takes as input $\mathbf{z}_{i+1}$, $c'_i \mathbf{s}$, and i . For convenience, write $\mathbf{z}_{i+1} \triangleq \mathbf{x}$ and $c'_i \mathbf{s} \triangleq \mathbf{r}$, and treat them as vectors. Let $d_1 = \ell n$, $d_2 = kn$, and $d = (1 + \ell + k)n$. The d -dimensional vector $\mathbf{x}$ is divided into three consecutive parts: $\mathbf{x}_0 \in \mathbb{Z}^n$, $\mathbf{x}_1 \in \mathbb{Z}^{d_1}$, and $\mathbf{x}_2 \in \mathbb{Z}^{d_2}$;

Algorithm 2 rANS encoding $\text{rANS.Encode}(\mathbf{s})$

```

1: Parse  $\mathbf{s} = (s_1, \dots, s_m) \in S^m$ 
2:  $x_0 \leftarrow 0$ 
3: for  $i = 0$  to  $m - 1$  do
4:    $x_{i+1} \leftarrow \lfloor x_i / g(s_{i+1}) \rfloor \cdot 2^t + (x_i \bmod g(s_{i+1})) + CDF(s_{i+1})$ 
5: end for
6: return  $x_m$ 

```

Algorithm 3 rANS decoding $\text{rANS.Decode}(x)$

```

1:  $y_0 \leftarrow x$ 
2:  $i \leftarrow 0$ 
3: while  $y_i > 0$  do
4:    $s'_{i+1} \leftarrow D^{-1}(y_i \bmod 2^t)$ 
5:    $y_{i+1} \leftarrow \lfloor y_i / 2^t \rfloor \cdot g(s'_{i+1}) + (y_i \bmod 2^t) - CDF(s'_{i+1})$ 
6:    $i \leftarrow i + 1$ 
7: end while
8:  $m \leftarrow i - 1$ 
9: return  $(s'_m, \dots, s'_1)$ 

```

$\mathbf{r}$ is treated similarly. Let $\tau_i = \min(i\tau', \tau)$, and define a distribution $\mathcal{D}'_i$ obtained by taking $\overline{\mathcal{D}}_{a_0, b_0}^{B_0-i} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_i} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_i}$, shifting it by $\mathbf{r}$ with probability 1/2 and by $-\mathbf{r}$ with probability 1/2. Correspondingly, define another distribution with smaller support as

$$\mathcal{D}_i = \overline{\mathcal{D}}_{a_0, b_0}^{B_0-(i+1)} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_{i+1}} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_{i+1}}.$$

Denote the probability mass functions of these two distributions by $g'_i(\mathbf{x})$ and $g_i(\mathbf{x})$, respectively. Let

$$A_0^i = \sum_{y=-B_0+i}^{B_0-i} f_{a_0, b_0}(y), \quad A_1^i = \sum_{y=-B_1+\tau_i}^{B_1-\tau_i} f_{a_1, b_1}(y), \quad A_2^i = \sum_{y=-B_2+\tau_i}^{B_2-\tau_i} f_{a_2, b_2}(y), \quad A^i = (A_0^i)^n (A_1^i)^{d_1} (A_2^i)^{d_2}$$

$$g0(\mathbf{x}_0) = g_{a_0, b_0}(\mathbf{x}_0), \quad g1(\mathbf{x}_1) = g_{a_1, b_1}(\mathbf{x}_1), \quad g2(\mathbf{x}_2) = g_{a_2, b_2}(\mathbf{x}_2), \quad g(\mathbf{x}) = g0(\mathbf{x}_0)g1(\mathbf{x}_1)g2(\mathbf{x}_2).$$

Then, by Appendix A, the probability mass function of $\mathbf{x}_0 \sim \overline{\mathcal{D}}_{a_0, b_0}^{B_0-i}$ is $\frac{g0(\mathbf{x}_0)}{(A_0^i)^n}$, that of $\mathbf{x}_1 \sim \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_i}$ is $\frac{g1(\mathbf{x}_1)}{(A_1^i)^{d_1}}$, and that of $\mathbf{x}_2 \sim \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_i}$ is $\frac{g2(\mathbf{x}_2)}{(A_2^i)^{d_2}}$. Let the support of $\mathbf{x} \sim \mathcal{D}_i$ be C_i . Then for any $\mathbf{x} \sim \mathcal{D}_i$ we have

$$|(\mathbf{x}_0)_j| \leq B_0 - i - 1, \quad |(\mathbf{x}_1)_j| \leq B_1 - \tau_{i+1}, \quad |(\mathbf{x}_2)_j| \leq B_2 - \tau_{i+1}.$$

The probability of $\mathbf{x} \sim \mathcal{D}_i$ is

$$g_i(\mathbf{x}) = \frac{g0(\mathbf{x}_0)g1(\mathbf{x}_1)g2(\mathbf{x}_2)}{(A_0^{i+1})^n (A_1^{i+1})^{d_1} (A_2^{i+1})^{d_2}} = \frac{g(\mathbf{x})}{A^{i+1}}.$$

Let the support of $\mathbf{x} \sim \mathcal{D}'_i$ be C'_i ; then $C_i \subseteq C'_i$, and when $\mathbf{x} \in C_i$, the probability mass function of $\mathbf{x} \sim \mathcal{D}'_i$ is

$$g'_i(\mathbf{x}) = \frac{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})}{2A^i}.$$

The input to the *Reject* algorithm is $\mathbf{x} \sim \mathcal{D}'_i$; the *Reject* algorithm (see Algorithm 4) outputs False with probability close to $p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})}$, where

$$p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})} = \begin{cases} \frac{2A^i}{M_i A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})}, & \mathbf{x} \in C_i \\ 0, & \mathbf{x} \in C'_i \setminus C_i \end{cases}$$

When $\mathbf{x} \in C'_i \setminus C_i$, $p_i = 0$, i.e. the *Reject* algorithm always returns True; step 1 of Algorithm 4 implements this rejection. Therefore, in the following we mainly consider the case $\mathbf{x} \in C_i$, and describe how the *Reject* algorithm approximates p_i without any floating-point operation, and prove that the relative error of this approximation is less than $2^{-L+4+\log d}$.

Let $M_i = \frac{2^L}{m_i}$ be such that $p_i \leq 1$ always holds, where m_i and $L = 64$ are integers. For $\mathbf{x} \in C_i$ we have

$$p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})} = \frac{m_i A^i}{2^{L-1} A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})} = \frac{m_i (A_0^i)^n (A_1^i)^{d_1} (A_2^i)^{d_2}}{2^{L-1} (A_0^{i+1})^n (A_1^{i+1})^{d_1} (A_2^{i+1})^{d_2}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})}.$$

Furthermore, define the following four lists of positive integers:

$$\begin{aligned} list_0 &= [m_i] \cup [A_p^i]_{p,j} \cup [f_{a_p, b_p}((\mathbf{x}_p)_j)]_{p,j}, \\ list_1 &= [2^{L-1}] \cup [A_p^{i+1}]_{p,j}, \\ list_2 &= [f_{a_p, b_p}((\mathbf{x}_p + \mathbf{r}_p)_j)]_{p,j}, \\ list_3 &= [f_{a_p, b_p}((\mathbf{x}_p - \mathbf{r}_p)_j)]_{p,j}. \end{aligned}$$

By Appendix A, $g(\mathbf{x}) = \prod_{j=1}^n f_{a_0, b_0}((\mathbf{x}_0)_j) \prod_{j=1}^{d_1} f_{a_1, b_1}((\mathbf{x}_1)_j) \prod_{j=1}^{d_2} f_{a_2, b_2}((\mathbf{x}_2)_j)$. Therefore,

$$p_i = \frac{\prod_{y \in list_0} y}{\prod_{y \in list_1} y (\prod_{y \in list_2} y + \prod_{y \in list_3} y)}. \quad (2)$$

The goal of the *Reject* algorithm (see Algorithm 4) is to compute an approximation $\frac{N}{D}$ of p_i without any floating-point operation, and to return False with probability $\frac{N}{D}$. In Equation Equation (2), every y is an integer, so both the numerator and the denominator are integers; one can therefore use random bits and rejection sampling to output False with probability $\frac{N}{D}$ without floating-point operations. However, a key drawback of this method is that the numerator and denominator computations involve the product of many integers; computing them exactly would involve very large integers and significantly reduce efficiency. For this reason, we use Algorithm 5 to approximate the product operations so as to avoid very large integers, ensuring that all multiplicands are integers of at most L bits, thereby improving computational efficiency. Specifically, given a list of positive integers $list$, the main idea of Algorithm 5 is to first truncate each list element to at most L bits (this step is done by precomputation), and then iteratively compute the product of the first i elements of $list$. If this product exceeds L bits, it is truncated to L bits,

and then multiplied by $list[i + 1]$. Finally, [Algorithm 5](#) outputs a large integer N' of at most L bits and an integer coe , such that $2^{coe}N'$ approximates $\prod_{x \in list} x$. Lines 1–12 of [Algorithm 4](#) call [Algorithm 5](#) several times to obtain the approximation $\frac{N}{D}$ of p_i , and lines 13–21 use random-bit sampling and rejection so that [Algorithm 4](#) outputs False with probability $\frac{N}{D}$. By [Lemma B.2](#), the relative error of $\frac{N}{D}$ with respect to p_i is less than $2^{-L+4+\log d}$.

Algorithm 4 *Reject*($\mathbf{x}, \mathbf{r}, j$)

```

1: if  $\|\mathbf{z}_{i+1}^0\|_\infty > B_0 - (i + 1)$  or  $\|\mathbf{z}_{i+1}^1\|_\infty > B_1 - \tau_{i+1}$  or  $\|\mathbf{z}_{i+1}^2\|_\infty > B_2 - \tau_{i+1}$  then
2:   return True  $\triangleright \mathbf{z}_{i+1} \in C'_i \setminus C_i$ 
3: end if
4:  $N, coe_0 = Approx(list_0)$ 
5:  $N_1, coe_1 = Approx(list_1)$ 
6:  $N_2, coe_2 = Approx(list_2)$ 
7:  $N_3, coe_3 = Approx(list_3)$ 
8:  $coe_4 = \min(coe_2, coe_3)$ 
9:  $D, coe_5 = Approx([N_1, 2^{coe_2 - coe_4}N_2 + 2^{coe_3 - coe_4}N_3])$ 
10:  $coe = coe_0 - coe_1 - coe_4 - coe_5$ 
11: if  $coe \geq 0$  then
12:    $N = 2^{coe}N$ 
13: else
14:    $D = 2^{-coe}D$ 
15: end if
16:  $bits = \text{len}(D)$   $\triangleright \text{len}(x)$  denotes the bit length of  $x$ 
17:  $r \leftarrow [2^{bits}]$ 
18: if  $r \geq D$  then
19:   Goto step 14
20: end if
21: if  $r \geq N$  then
22:   return True
23: else
24:   return False
25: end if

```

3 Signature Scheme

Based on the preliminaries and auxiliary functions in [Section 2](#), this section describes the CS signature scheme. [Section 3.1](#) describes the core algorithm of CS—the rejection sampling algorithm. [Section 3.2](#) describes the simplified CS signature scheme in order to highlight its design rationale and innovations. [Section 3.3](#) describes the full CS signature scheme by adding

Algorithm 5 *Approx(list)*

```

1:  $d' = \text{len}(\text{list})$ 
2:  $N' = 1$ 
3:  $\text{coe} = 0$ 
4: for  $i$  from 0 to  $d' - 1$  do
5:    $s = \text{len}(\text{list}[i]) - L$   $\triangleright \text{len}(x)$  denotes the bit length of  $x$ 
6:   if  $s > 0$  then
7:      $\text{list}[i] \gg s$ 
8:      $\text{coe} += s$ 
9:   end if
10:   $N' = N' \cdot \text{list}[i]$ 
11:   $s = \text{len}(N') - L$ 
12:  if  $s > 0$  then
13:     $N' \gg s$   $\triangleright$  right-shift  $N'$  by  $s$  bits
14:     $\text{coe} += s$ 
15:  end if
16: end for
17: return  $N', \text{coe}$ 

```

the generation and use of randomness, public-key compression, and signature compression on the basis of the simplified scheme. Section 3.4 proves the correctness of the CS signature scheme and gives its signature repetition count.

3.1 Rejection Sampling Algorithm

A key algorithm in FS lattice-based signature schemes is the rejection sampling algorithm, and the same holds for the CS scheme. The goal of the CS scheme is to implement a bimodal FS lattice-based signature scheme without floating-point operations throughout the whole process, which makes it significantly different from traditional bimodal FS rejection sampling algorithms. Formally, our rejection sampling algorithm (Algorithm 6) is a combination of the traditional bimodal rejection sampling algorithm [DDLL13] and the iterative rejection sampling algorithm [Gar25]. Specifically, the CS rejection sampling algorithm takes the iterative rejection sampling algorithm [Gar25] and sets the probabilities of taking $\mathbf{z} = \mathbf{y} + \mathbf{c}s$ and $\mathbf{z} = \mathbf{y} - \mathbf{c}s$ to $\frac{1}{2}$ each, as in [DDLL13]. In addition, by introducing a parameter τ' , the iterative rejection becomes more flexible, and by adjusting this parameter we obtain more efficient CS parameters.

Let the non-zero positions of c be $j_0, \dots, j_{\tau-1}$ (the algorithm parameters such as τ are given in Table 3). c'_i agrees with c at positions $j_{i\tau'}, \dots, j_{\min((i+1)\tau'-1, \tau-1)}$ (the Hamming weight of c'_i is at most τ') and is zero elsewhere; therefore $c = c'_0 + \dots + c'_{N-1}$. Taking as input $\mathbf{s}, \mathbf{y}, c, \tau'$, Algorithm 6 first obtains the c'_i as described above and performs $N = \lceil \frac{\tau}{\tau'} \rceil$ iterations. In each iteration, it first computes $\mathbf{z}_{i+1} = \mathbf{z}_i \pm c'_i \mathbf{s}$ (line 5) and determines whether to reject it at line 9. Since the

three components of $\mathbf{y} = (\mathbf{y}_0, \mathbf{y}_1^\top, \mathbf{y}_2^\top)^\top$ are drawn from the distributions $\mathcal{D}_{a_0, b_0}^{\mathcal{R}}, \mathcal{D}_{a_1, b_1}^{\mathcal{R}^\ell}, \mathcal{D}_{a_2, b_2}^{\mathcal{R}^k}$, respectively, by the definitions in Section 2.7 the $\mathbf{z}_{i+1}$ after line 8 follows $\mathcal{D}'_i$, which is obtained by taking $\overline{\mathcal{D}}_{a_0, b_0}^{B_0-i} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_i} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_i}$, shifting it by $\mathbf{r} = c'_i \mathbf{s}$ with probability 1/2 and by $-\mathbf{r}$ with probability 1/2, where $\tau_i = \min(i\tau', \tau)$. Further, by the definitions in Section 2.7, the input $\mathbf{z}_{i+1}$ of the *Reject* algorithm satisfies

$$\mathbf{z}_{i+1} \sim \mathcal{D}'_i,$$

and its probability mass function is $g'_i(\mathbf{z}_{i+1})$. Using the definitions in Section 2.7, let

$$\mathcal{D}_i = \overline{\mathcal{D}}_{a_0, b_0}^{B_0-(i+1)} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_{i+1}} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_{i+1}}$$

and let $g_i(\mathbf{z}_{i+1})$ be the probability mass function of $\mathbf{z}_{i+1} \sim \mathcal{D}_i$. Then, if the *Reject* algorithm in line 9 (see Algorithm 4) outputs False with probability

$$p_i = \frac{g_i(\mathbf{z}_{i+1})}{M_i g'_i(\mathbf{z}_{i+1})} \quad (3)$$

(where M_i is a constant such that $p_i \leq 1$ always holds), by Lemma B.4 the non- $\perp$ value of $\mathbf{z}_{i+1}$ follows the distribution $\mathcal{D}_i$ and the repetition count of line 9 in the i -th iteration is M_i . Thus, the *Reject* algorithm rejects the above $\mathbf{z}_{i+1}$ to a truncated Centered Multinomial Distribution $\mathcal{D}_i$ that is independent of the secret key $\mathbf{s}$, c'_i , and bit_i . However, the exact computation of p_i involves the product of many integers; to avoid very large integer operations, Algorithm 4 outputs False with probability $\frac{N}{D}$. By Lemma B.2, the relative error between p_i and $\frac{N}{D}$ is less than $2^{-L+4+\log d}$ (where $d = n(k+\ell+1)$); further, by Lemma B.3, this approximation reduces the concrete security strength of the CS scheme by at most 1 bit.

Algorithm 6 Rejection sampling algorithm *RejectSample*($\mathbf{s}, \mathbf{y}, c, \tau'$)

```

1:  $N = \lceil \frac{\tau}{\tau'} \rceil$ 
2: if  $\|\mathbf{y}_0\|_\infty > B_0$  or  $\|\mathbf{y}_1\|_\infty > B_1$  or  $\|\mathbf{y}_2\|_\infty > B_2$  then
3:   return  $\perp$ 
4: end if
5:  $\mathbf{z}_0 = \mathbf{y}$ 
6: for  $i = 0$  to  $N - 1$  do
7:    $bit_i \leftarrow \{0, 1\}$ 
8:    $\mathbf{z}_{i+1} = \mathbf{z}_i + (-1)^{bit_i} c'_i \mathbf{s}$   $\triangleright$   $\mathbf{z}_{i+1}$  is obtained by shifting  $\mathbf{z}_i$  by  $c'_i \mathbf{s}$  or  $-c'_i \mathbf{s}$  with probability 1/2
   each
9:   if Reject( $\mathbf{z}_{i+1}, c'_i \mathbf{s}, i$ ) == True then  $\triangleright$  outputs False with probability  $\frac{N}{D} \approx p_i = \frac{g_i(\mathbf{z}_{i+1})}{M_i g'_i(\mathbf{z}_{i+1})}$ 
10:    return  $\perp$   $\triangleright \mathbf{z}_{i+1} \sim \mathcal{D}_i$ 
11:   end if
12: end for
13: return  $\mathbf{z}_N, c_r = \sum_{i=0}^{N-1} (-1)^{bit_i} c'_i$ 
```

3.2 Simplified CS Signature Scheme

The simplified key generation algorithm (see [Algorithm 7](#)) follows the key-generation idea of the HAETAE scheme [[CHE⁺24](#)] and uses the MLWE hard problem; that is, the public key of the CS signature scheme is $(\mathbf{A}', \mathbf{b})$, where $\mathbf{b} = \mathbf{A}'\mathbf{s}_1 + \mathbf{s}_2$. In line 5 of the algorithm, the first polynomial component of the polynomial vector $\mathbf{j} \in \mathcal{R}^k$ is 1, and the other polynomial components are 0. The $\mathbf{A}, \mathbf{s}$ output by this key generation algorithm satisfy

$$\mathbf{A}\mathbf{s} = q\mathbf{j} - 2\mathbf{b} + 2\mathbf{A}'\mathbf{s}_1 + 2\mathbf{s}_2 = q\mathbf{j} \bmod 2q.$$

Algorithm 7 Simplified key generation algorithm *KeyGenSimple*

- 1: $\mathbf{A}' \leftarrow \mathcal{R}_q^{k \times \ell}$
 - 2: $\mathbf{s}_1 \leftarrow \mathcal{S}^\ell$
 - 3: $\mathbf{s}_2 \leftarrow \mathcal{S}^k$
 - 4: $\mathbf{b} = \mathbf{A}'\mathbf{s}_1 + \mathbf{s}_2 \in \mathcal{R}_q^k$
 - 5: $\mathbf{A} = (q\mathbf{j} - 2\mathbf{b} | 2\mathbf{A}' | 2\mathbf{I}_k) \in \mathcal{R}_{2q}^{k \times (1+\ell+k)}$
 - 6: $\mathbf{s} = (1, \mathbf{s}_1^\top, \mathbf{s}_2^\top)^\top \in \mathcal{R}^{k+\ell+1}$
 - 7: **return** $pk = (\mathbf{A}', \mathbf{b}), sk = \mathbf{s}$
-

Our goal is to design a compact bimodal FS lattice-based signature scheme that is free of floating-point operations throughout the whole process; the core of the design lies in the differences in the signing algorithm. The signing process of bimodal FS lattice-based signature schemes often involves floating-point operations, including the generation of the mask vector $\mathbf{y}$ and the computation of the rejection probability for the signature $\mathbf{z}$. To avoid floating-point operations in these two steps, the CS signature scheme is designed as follows:

1. The mask vector $\mathbf{y}$ uses a simple Centered Multinomial Distribution that is easy to sample by integer operations.
2. Based on Rényi divergence, we propose a method that computes only an approximation of the signature rejection probability through integer operations.

Through the above designs, the CS signature scheme is free of floating-point operations. Moreover, we extend the coefficient space of the challenge polynomial c in bimodal FS signature schemes from $\{0, 1\}$ to $\{0, \pm 1\}$, which reduces the Hamming weight of the challenge polynomial and makes the CS signature scheme more compact. In order to improve the efficiency of the signing algorithm, the CS signature scheme follows the idea of iterative bimodal [[Gar25](#)] and improves the flexibility of the iterative bimodal design by introducing a new parameter τ' .

The first three lines of the simplified signing algorithm (see [Algorithm 8](#)) are mainly used to generate the mask vector $\mathbf{y}$ following the centered multinomial distribution. The main difficulty of [Algorithm 8](#) lies in line 6, where the rejection sampling algorithm *RejectSample* generates the signature $\mathbf{z} = (\mathbf{z}_0, \mathbf{z}_1^\top, \mathbf{z}_2^\top)^\top \in \mathcal{R} \times \mathcal{R}^\ell \times \mathcal{R}^k$. In order to implement the rejection sampling

algorithm efficiently and securely, we approximate the signature rejection probability without any floating-point operation throughout the whole process, and prove by controlling the relative error under Rényi divergence that this approximation reduces the concrete security strength of the scheme by at most 1 bit (see [Algorithm 6](#)).

Algorithm 8 Simplified signing algorithm $SignSimple(\mathbf{A}, \mathbf{s}, M)$

```

1:  $\mathbf{y}_0 \leftarrow \mathcal{D}_{a_0, b_0}^{\mathcal{R}}$ 
2:  $\mathbf{y}_1 \leftarrow \mathcal{D}_{a_1, b_1}^{\mathcal{R}^\ell}$ 
3:  $\mathbf{y}_2 \leftarrow \mathcal{D}_{a_2, b_2}^{\mathcal{R}^k}$ 
4:  $\mathbf{y} = (\mathbf{y}_0, \mathbf{y}_1^\top, \mathbf{y}_2^\top)^\top \in \mathcal{R}^{1+\ell+k}$ 
5:  $\mathbf{w} = \mathbf{A}\mathbf{y} \bmod 2q$ 
6:  $\tilde{c} = H(M, \mathbf{w}) \in \{0, 1\}^{2\lambda}$ 
7:  $c = SampleInBall(\tilde{c}, \tau)$ 
8:  $\mathbf{z}, c_r = RejectSample(\mathbf{s}, \mathbf{y}, c, \tau')$ 
9: if  $\mathbf{z} = \perp$  then
10:   Goto step 1
11: end if
12: return  $(\mathbf{z}, \tilde{c})$  ▷ rANS encoding

```

The signature verification algorithm is relatively simple (see [Algorithm 9](#)). Like traditional bimodal FS lattice-based signature schemes, the main difference is that the CS signature verification algorithm bounds the infinity norm of the signature $\mathbf{z}$, rather than the 2-norm.

Algorithm 9 Simplified signature verification algorithm $VerifySimple(\mathbf{A}, M, \mathbf{z}, \tilde{c})$

```

1:  $c' = SampleInBall(\tilde{c}, \tau)$ 
2:  $\mathbf{w}' = \mathbf{A}\mathbf{z} - qc'\mathbf{j} \bmod 2q$ 
3: if  $\|\mathbf{z}_0\|_\infty > B_0 - \lceil \frac{\tau}{\tau'} \rceil$  or  $\|\mathbf{z}_1\|_\infty > B_1 - \tau$  or  $\|\mathbf{z}_2\|_\infty > B_2 - \tau$  or  $\tilde{c} \neq H(M, \mathbf{w}')$  then
4:   Reject
5: else
6:   Accept
7: end if

```

3.3 Full CS Signature Scheme

The key generation algorithm [Algorithm 10](#) builds on the simplified key generation algorithm [Algorithm 7](#) and compresses the public key: the $\mathbf{b}$ in the public key is replaced by $\mathbf{b}_1$, where $\mathbf{b}_1 = HighBits(\mathbf{b}, \beta)$ and $\mathbf{b}_0 = LowBits(\mathbf{b}, \beta)$, with β a power of 2; then we have

$$\mathbf{A}\mathbf{s} = q\mathbf{j} + 2\mathbf{b}_0 \bmod 2q.$$

In addition, it adds the generation of randomness and the generation of the random elements $\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2$, replaces the $\mathbf{A}'$ in the public key pk with the seed ρ_0 , and includes $K, tr, \rho_0, \mathbf{b}_0$ in the

secret key sk .

Algorithm 10 Key generation algorithm $KeyGen(1^\lambda)$

- 1: $\rho \leftarrow \{0, 1\}^{2\lambda}$
 - 2: $(\rho_0, \rho_1, K) := H(\rho, k, l) \in \{0, 1\}^\lambda \times \{0, 1\}^{2\lambda} \times \{0, 1\}^\lambda$
 - 3: $\mathbf{A}' := ExpandA(\rho_0) \in \mathcal{R}_q^{k \times \ell}$
 - 4: $(\mathbf{s}_1^T, \mathbf{s}_2^T)^\top := ExpandS(\rho_1) \in \mathcal{S}^\ell \times \mathcal{S}^k$
 - 5: $\mathbf{b} := \mathbf{A}'\mathbf{s}_1 + \mathbf{s}_2 \in \mathcal{R}_q^k$
 - 6: $\mathbf{A} = (q\mathbf{j} - 2\beta\mathbf{b}_1 | 2\mathbf{A}' | 2\mathbf{I}_k) \in \mathcal{R}_{2q}^{k \times (k+\ell+1)}$
 - 7: $\mathbf{s} = (1, \mathbf{s}_1^\top, \mathbf{s}_2^\top)^\top \in \mathcal{R}^{k+\ell+1}$
 - 8: $pk = (\rho_0, \mathbf{b}_1)$
 - 9: $tr := H(pk) \in \{0, 1\}^{2\lambda}$
 - 10: $sk = (\mathbf{s}_1, \mathbf{s}_2, K, tr, \rho_0, \mathbf{b}_0)$
 - 11: **return** pk, sk
-

The signing algorithm [Algorithm 11](#) builds on the simplified signing algorithm [Algorithm 8](#). It uses compression techniques to replace the ring vector $\mathbf{z}_2$ in the signature by the hint $\mathbf{h}$, and uses rANS encoding to encode the ring element $\mathbf{z}_0$ and the ring vector $\mathbf{z}_1$, thereby reducing the signature size. In addition, it adds the generation of the matrix $\mathbf{A}$ and the mask vector $\mathbf{y}$. Because the key generation process compresses the public key, the computation of the vector $\mathbf{h}$ in line 18 involves $\mathbf{b}_0$ from the secret key.

Algorithm 11 Signing algorithm $Sign(sk, M)$

```

1:  $\mathbf{A}' := ExpandA(\rho_0) \in \mathcal{R}_q^{k \times \ell}$ 
2:  $\mathbf{A} = (q\mathbf{j} - 2\beta\mathbf{b}_1 | 2\mathbf{A}' | 2\mathbf{I}_k) \in \mathcal{R}_{2q}^{k \times (1+\ell+k)}$ 
3:  $\mu := H(tr, M) \in \{0, 1\}^{2\lambda}$ 
4:  $\kappa := 0$ 
5:  $\rho' := H(K, \mu) \in \{0, 1\}^{2\lambda}$  (randomized signing:  $\rho' \leftarrow \{0, 1\}^{2\lambda}$ )
6:  $\mathbf{y}_0 := ExpandMask(\rho', \kappa, a_0, b_0) \sim \mathcal{D}_{a_0, b_0}^{\mathcal{R}}$ 
7:  $\mathbf{y}_1 := ExpandMask(\rho', \kappa, a_1, b_1) \sim \mathcal{D}_{a_1, b_1}^{\mathcal{R}^\ell}$ 
8:  $\mathbf{y}_2 := ExpandMask(\rho', \kappa, a_2, b_2) \sim \mathcal{D}_{a_2, b_2}^{\mathcal{R}^k}$ 
9:  $\mathbf{y} = (\mathbf{y}_0, \mathbf{y}_1^\top, \mathbf{y}_2^\top)^\top \in \mathcal{R}^{k+\ell+1}$ 
10:  $\mathbf{w} := \mathbf{A}\mathbf{y} \bmod 2q$ 
11:  $\tilde{c} := H(\mu, HighBits^h(\mathbf{w}, \alpha), LSB(\mathbf{y}_0)) \in \{0, 1\}^{2\lambda}$ 
12:  $c = SampleInBall(\tilde{c}, \tau)$ 
13:  $\mathbf{z}, c_r := RejectSample(\mathbf{s}, \mathbf{y}, c, \tau')$ 
14: if  $\mathbf{z} = \perp$  then
15:    $\kappa \leftarrow \kappa + 1$ 
16:   Goto step 6
17: end if
18:  $\mathbf{h} := HighBits^h(\mathbf{w}, \alpha) - HighBits^h(\mathbf{w} - 2\mathbf{z}_2 + 2c_r\mathbf{b}_0, \alpha) \bmod \pm \frac{2(q-1)}{\alpha}$ 
19:  $\sigma_0 := \text{rANS.Encode}(\mathbf{z}_0)$   $\triangleright \mathbf{z}_0 \in \mathcal{R}$ 
20:  $\sigma_1 := \text{rANS.Encode}(\mathbf{z}_1)$   $\triangleright \mathbf{z}_1 \in \mathcal{R}^\ell$ 
21: return  $\sigma = (\sigma_0, \sigma_1, \mathbf{h}, \tilde{c})$ 

```

Based on the signing algorithm [Algorithm 11](#), the verification algorithm [Algorithm 12](#) also makes corresponding adjustments on the basis of the simplified verification algorithm [Algorithm 9](#). In line 13 of [Algorithm 12](#), $B'_2 = B_2 - \tau + \frac{\alpha}{4} + 1 + \tau \frac{\beta}{2}$.

Algorithm 12 Signature verification algorithm $Verify(pk, M, \sigma)$

```

1:  $\mathbf{A}' := ExpandA(\rho_0)$ 
2:  $\mathbf{A}_0 = q\mathbf{j} - 2\beta\mathbf{b}_1 \in \mathcal{R}_{2q}^k$ 
3:  $\mathbf{A}_1 = 2\mathbf{A}' \in \mathcal{R}_{2q}^{k \times l}$ 
4:  $\mu' := H(H(pk), M)$ 
5:  $\mathbf{z}'_0 := \text{rANS.Decode}(\sigma_0)$ 
6:  $\mathbf{z}'_1 := \text{rANS.Decode}(\sigma_1)$ 
7:  $c' := SampleInBall(\tilde{c}, \tau)$ 
8:  $\mathbf{v} := \mathbf{A}_0\mathbf{z}'_0 + \mathbf{A}_1\mathbf{z}'_1 - qc'\mathbf{j} \bmod 2q$ 
9:  $\mathbf{w}' := HighBits^h(\mathbf{v}, \alpha) + \mathbf{h} \bmod \frac{2(q-1)}{\alpha}$ 
10:  $w' := LSB(\mathbf{z}'_0 - c')$ 
11:  $\mathbf{z}'_2 := (\alpha \cdot \mathbf{w}' + w'\mathbf{j} - \mathbf{v} \bmod 2q)/2 \bmod \pm q$ 
12: if  $\|\mathbf{z}'_0\|_\infty > B_0 - \lceil \frac{\tau}{\tau'} \rceil$  or  $\|\mathbf{z}'_1\|_\infty > B_1 - \tau$  or  $\|\mathbf{z}'_2\|_\infty > B'_2$  or  $\tilde{c} \neq H(\mu', \mathbf{w}', w')$  then
13:   Reject
14: else
15:   Accept
16: end if

```

3.4 Correctness and Signature Repetition Count

This subsection analyses the correctness and signature repetition count of the full CS signature scheme.

Theorem 3.1. *For any key pair $(pk, sk) \leftarrow KeyGen(1^\lambda)$, any message M , and any signature σ output by $Sign(sk, M)$, the signature verification algorithm $Verify(pk, M, \sigma)$ always accepts.*

Proof. We first prove that the equality $\tilde{c} = H(\mu', \mathbf{w}', w')$ in line 13 of the verification algorithm always holds. For a signature $\sigma = (\sigma_0, \sigma_1, \mathbf{h}, \tilde{c})$ output by $Sign(sk, M)$, the symbols in the signing and verification algorithms satisfy

$$\mu' = \mu, \quad \mathbf{z}'_0 = \mathbf{z}_0, \quad \mathbf{z}'_1 = \mathbf{z}_1, \quad c' = SampleInBall(\tilde{c}, \tau) = c.$$

Let $N = \lceil \frac{\tau}{\tau'} \rceil$ and $c_r = \sum_{i=0}^{N-1} (-1)^{bit_i} c'_i$. Then the value $\mathbf{v}$ in the verification algorithm satisfies

$$\begin{aligned} \mathbf{v} &= \mathbf{A}_0 \mathbf{z}_0 + \mathbf{A}_1 \mathbf{z}_1 - qc\mathbf{j} \bmod 2q \\ &= \mathbf{A}\mathbf{z} - \mathbf{A}_2 \mathbf{z}_2 - qc\mathbf{j} \bmod 2q \\ &= \mathbf{A}(\mathbf{y} + c_r \mathbf{s}) - 2\mathbf{z}_2 - qc\mathbf{j} \bmod 2q \\ &= \mathbf{w} + c_r(q\mathbf{j} + 2\mathbf{b}_0) - 2\mathbf{z}_2 - qc\mathbf{j} \bmod 2q \\ &= \mathbf{w} - 2\mathbf{z}_2 + 2c_r \mathbf{b}_0 \bmod 2q. \end{aligned}$$

Further,

$$\begin{aligned} \mathbf{w}' &= HighBits^h(\mathbf{v}, \alpha) + \mathbf{h} \bmod \frac{2(q-1)}{\alpha} \\ &= HighBits^h(\mathbf{w} - 2\mathbf{z}_2 + 2c_r \mathbf{b}_0, \alpha) + \mathbf{h} \bmod \frac{2(q-1)}{\alpha} \\ &= HighBits^h(\mathbf{w}, \alpha). \end{aligned}$$

Moreover, since $w' = LSB(\mathbf{z}_0 - c) = LSB(\mathbf{y}_0 + c_r - c) = LSB(\mathbf{y}_0)$, we have $H(\mu', \mathbf{w}', w') = \tilde{c}$.

Next we prove that the bound checks in line 13 of the verification algorithm always hold. By the iterative rejection process of Algorithm 6, $\|\mathbf{z}'_0\|_\infty \leq B_0 - N$, $\|\mathbf{z}'_1\|_\infty \leq B_1 - \tau$, and $\|\mathbf{z}_2\|_\infty \leq B_2 - \tau$ hold. By

$$LSB(\mathbf{w}) = LSB((q\mathbf{j} - 2\beta\mathbf{b}_1)\mathbf{y}_0 + 2\mathbf{A}'\mathbf{y}_1 + 2\mathbf{y}_2) = LSB(\mathbf{y}_0)\mathbf{j},$$

and the properties of the hint high/low bits functions in Section 2,

$$\begin{aligned} \mathbf{z}'_2 &= (\alpha \cdot \mathbf{w}' + w'\mathbf{j} - \mathbf{v} \bmod 2q)/2 \bmod \pm q \\ &= (\alpha \cdot HighBits^h(\mathbf{w}, \alpha) + LSB(\mathbf{y}_0)\mathbf{j} - (\mathbf{w} - 2\mathbf{z}_2 + 2c_r \mathbf{b}_0) \bmod 2q)/2 \bmod \pm q \\ &= \mathbf{z}_2 - c_r \mathbf{b}_0 + (-LowBits^h(\mathbf{w}, \alpha) + LSB(\mathbf{w}))/2 \bmod \pm q. \end{aligned}$$

The last equality holds because, under our parameter settings, $\|2\mathbf{z}_2 - 2c_r \mathbf{b}_0 - LowBits^h(\mathbf{w}, \alpha) + LSB(\mathbf{w})\|_\infty \leq 2q$. Since $LowBits^h(\mathbf{w}, \alpha) \in [-\frac{\alpha}{2} - 2, \frac{\alpha}{2})$, we have $\|\mathbf{z}'_2 - \mathbf{z}_2\|_\infty \leq \frac{\alpha}{4} + 1 + \tau \frac{\beta}{2}$, and further $\|\mathbf{z}'_2\|_\infty \leq \|\mathbf{z}_2\|_\infty + \|\mathbf{z}'_2 - \mathbf{z}_2\|_\infty \leq B'_2$. This completes the proof of correctness of the CS signature scheme. $\square$

The signature repetition count of the CS scheme mainly comes from two parts: the bound rejection (Algorithm 6 line 2) and the probabilistic rejection (Algorithm 6 line 9). The distributions before and after the bound rejection are

$$\mathcal{D}_{a_0, b_0} \times \mathcal{D}_{a_1, b_1} \times \mathcal{D}_{a_2, b_2}, \quad \overline{\mathcal{D}}_{a_0, b_0}^{B_0} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2}.$$

Based on the definitions in Section 2.7, the bound rejection probability is

$$p' = \frac{A^0}{2^{2(na_0b_0 + d_1a_1b_1 + d_2a_2b_2)}} \triangleq \frac{A_0}{A}.$$

Using the definitions in Section 2.7, and letting $f_{a_0, b_0} = f_0$, $f_{a_1, b_1} = f_1$, $f_{a_2, b_2} = f_2$, Theorem 3.2 gives the repetition count of Algorithm 6 line 9 in the i -th iteration, and gives the signature repetition count of the CS scheme.

Theorem 3.2. *For any $\mathbf{s}$ and c , under the CS parameters, the repetition count of Algorithm 6 line 9 in the i -th iteration is*

$$M_i = \frac{A^i g(\mathbf{x}^i)}{A^{i+1} \sqrt{g_i(\mathbf{x}^i + \mathbf{r}^i) g(\mathbf{x}^i - \mathbf{r}^i)}},$$

where $\mathbf{r}^i = (\mathbf{r}_0^i, \mathbf{r}_1^i, \mathbf{r}_2^i)$, $(\mathbf{r}_0^i)_j = 1$, $(\mathbf{r}_1^i)_j = (\mathbf{r}_2^i)_j = \tau''$, $\tau'' = \min(\tau', \tau - i\tau')$, $\mathbf{x}^i = (\mathbf{x}_0^i, \mathbf{x}_1^i, \mathbf{x}_2^i)$, and $(\mathbf{x}_p^i)_j$ is the maximum point of the function $\frac{f_p^2(x)}{f_p(x+(\mathbf{r}_p^i)_j)f_p(x-(\mathbf{r}_p^i)_j)}$. Further, the repetition count M of the CS scheme is

$$\frac{A}{A^N} \prod_{i=0}^{N-1} \frac{g(\mathbf{x}^i)}{\sqrt{g(\mathbf{x}^i + \mathbf{r}^i) g(\mathbf{x}^i - \mathbf{r}^i)}}.$$

Proof. By the definitions in Section 2.7, $p_i = \frac{2A^i}{M_i A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x}+\mathbf{r})+g(\mathbf{x}-\mathbf{r})}$. Further, by Lemma B.4, the repetition count of line 9 in the i -th iteration is M_i , where M_i is such that $p_i \leq 1$. Therefore,

$$M_i \geq \frac{2A^i}{A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x}+\mathbf{r})+g(\mathbf{x}-\mathbf{r})} \geq \frac{A^i}{A^{i+1}} \frac{g(\mathbf{x})}{\sqrt{g(\mathbf{x}+\mathbf{r})g(\mathbf{x}-\mathbf{r})}}.$$

Let

$$M'_i(\mathbf{x}, \mathbf{r}) = \frac{g(\mathbf{x})}{\sqrt{g(\mathbf{x}+\mathbf{r})g(\mathbf{x}-\mathbf{r})}},$$

By Lemma A.1, M'_i is an increasing function of $|\mathbf{r}_p|_j$, so for $\mathbf{r} = \mathbf{r}^i$, $M'_i(\mathbf{x}, \mathbf{r})$ attains its maximum $M'_i(\mathbf{x}, \mathbf{r}^i)$, abbreviated as $M'_i(\mathbf{x})$. Then $M'_i(\mathbf{x})$ can be decomposed into the product of d factors:

$$\prod_{j=1}^n \frac{f_0^2((\mathbf{x}_0)_j)}{f_0((\mathbf{x}_0)_j + (\mathbf{r}_0^i)_j) f_0((\mathbf{x}_0)_j - (\mathbf{r}_0^i)_j)} \prod_{j=1}^{d_1} \frac{f_1^2((\mathbf{x}_1)_j)}{f_1((\mathbf{x}_1)_j + (\mathbf{r}_1^i)_j) f_1((\mathbf{x}_1)_j - (\mathbf{r}_1^i)_j)} \prod_{j=1}^{d_2} \frac{f_2^2((\mathbf{x}_2)_j)}{f_2((\mathbf{x}_2)_j + (\mathbf{r}_2^i)_j) f_2((\mathbf{x}_2)_j - (\mathbf{r}_2^i)_j)}$$

Therefore, when each function $\frac{f_p^2(x)}{f_p(x+(\mathbf{r}_p^i)_j)f_p(x-(\mathbf{r}_p^i)_j)}$ attains its maximum point, the repetition count of the i -th iteration is

$$M_i = \frac{A^i g(\mathbf{x}^i)}{A^{i+1} \sqrt{g_i(\mathbf{x}^i + \mathbf{r}^i) g(\mathbf{x}^i - \mathbf{r}^i)}}.$$

Further, the total signature repetition count is

$$M = \frac{1}{p'} \prod_{i=0}^{N-1} \frac{A^i g(\mathbf{x}^i)}{A^{i+1} \sqrt{g_i(\mathbf{x}^i + \mathbf{r}^i) g(\mathbf{x}^i - \mathbf{r}^i)}} = \frac{A}{A^N} \prod_{i=0}^{N-1} \frac{g(\mathbf{x}^i)}{\sqrt{g(\mathbf{x}^i + \mathbf{r}^i) g(\mathbf{x}^i - \mathbf{r}^i)}}.$$

□

3.5 Parameter Sets and Selection

One feature of the CS scheme is that it has many parameters, which allows the scheme to adapt to different application scenarios by optimising the parameter configuration. The effects of the parameters on the scheme's properties are as follows:

1. The parameters that affect the entropy of the challenge polynomial are n and τ .
2. The main parameters that affect the hardness of the MLWE problem are n , q , ℓ , and η .
3. The main parameters that affect MSIS/BSTMSIS are n , q , k , B_i , β , and α .

Table 3: Parameter sets of the CS scheme

λ	Security parameter	128	256	512
n	Dimension of polynomial ring	256	512	512
q	Modulus	32257	64513	64513
k, ℓ	Rows/columns of ring matrix $\mathbf{A}_0$	3,3	3,3	6,5
η	Secret-key distribution parameter	1	1	1
a_0, b_0, B_0	Parameters related to $\mathbf{y}_0$	5,5,116	6,6,278	6,6,261
a_1, b_1, B_1	Parameters related to $\mathbf{y}_1$	10,9,2779	9,10,5385	12,11,12793
a_2, b_2, B_2	Parameters related to $\mathbf{y}_2$	6,9,1962	9,9,2569	8,11,9771
τ, τ'	Parameters related to the challenge polynomial	23,4	44,3	118,6
β, α	Public-key compression parameters, hint parameters	32,2016	128,8064	32,4032
Entropy of c		131.08	256.4	512.18
M	Signature repetition count	1.75	1.96	2.00

4. The main parameters that affect the signature repetition count are $n, k, \ell, a_i, b_i, B_i, \tau$, and τ' .
5. The main parameters that affect the scheme's sizes are $n, k, \ell, a_i, b_i, B_i, \beta$, and α .
6. The main parameters that affect the scheme's efficiency are $n, q, k, \ell, a_i, b_i, B_i, \tau$, and τ' .

We select parameters roughly as follows:

1. Choose n and τ so that the entropy reaches the target security strength λ , where n is a power of 2.
2. Under the premise that $2n \mid (q - 1)$ so that the NTT can be used to accelerate polynomial multiplication, choose q and ℓ so that the MLWE problem meets the security requirements, where q is a prime.
3. Choose k, a_i, b_i, B_i, τ , and τ' so that the signature repetition count is small (guaranteeing efficiency) and, when $\beta = 0$ and $\alpha = 0$, the MSIS/BSTMSIS problem meets the target security level. If no parameter choice satisfies these conditions, return to the first step and reselect n, q , and ℓ .
4. Choose β and α so that the MSIS/BSTMSIS problem meets the target security level while obtaining a more compact signature scheme. If no parameter choice satisfies these conditions, return to the third step and reselect parameters.

The above procedure can help us find parameters that meet the security requirements, but finding a compact and efficient parameter set still requires a large amount of experimentation and tuning. We give the parameter choices of the CS scheme at different security levels in Table 3. In addition, a parameter M_i is needed to denote the repetition count of Algorithm 6 line 9 in the i -th iteration; the formula for computing M_i is given in Theorem 3.2.

4 Security Analysis

4.1 Cryptographic Definitions

We first introduce the MLWE and MSIS hard problems over lattices.

Definition 4.1 (Decisional MLWE $_{n,q,k,\ell,\eta}$). *For positive integers q, k, ℓ, η and $\mathcal{R}$ of dimension n , we define the advantage of adversary $\mathcal{A}_1$ in solving the decisional MLWE $_{n,q,k,\ell,\eta}$ problem as*

$$\text{Adv}_{n,q,k,\ell,\eta}^{\text{MLWE}}(\mathcal{A}_1) = \left| \frac{\Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; \mathbf{b} \leftarrow \mathcal{R}_q^k; b \leftarrow \mathcal{A}_1(\mathbf{A}, \mathbf{b})]}{-\Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; (\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_\eta^\ell \times S_\eta^k; b \leftarrow \mathcal{A}_1(\mathbf{A}, \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2)]} \right|.$$

Definition 4.2 (MSIS $_{n,q,k,\ell,\gamma}$). *For positive integers q, k, ℓ , positive real number γ , and $\mathcal{R}$ of dimension n , we define the advantage of adversary $\mathcal{A}_2$ in solving the MSIS $_{n,q,k,\ell,\gamma}$ problem as*

$$\text{Adv}_{n,q,k,\ell,\gamma}^{\text{MSIS}}(\mathcal{A}_2) = \Pr \left[\begin{array}{c} 0 < \|\mathbf{y}\|_\infty < \gamma \wedge \\ (\mathbf{A}|\mathbf{I}_k) \cdot \mathbf{y} = 0 \bmod q \end{array} \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; \mathbf{y} \in \mathcal{R}_q^{k+\ell} \leftarrow \mathcal{A}_2(\mathbf{A}) \right].$$

Like other bimodal Fiat-Shamir lattice-based signature schemes, below we define a variant of the SelfTargetMSIS problem in ML-DSA, abbreviated as the BSTMSIS problem. However, unlike the BSTMSIS problem in other bimodal Fiat-Shamir lattice-based signature schemes, every coefficient of the challenge polynomial c in the CS signature scheme belongs to $\{0, \pm 1\}$.

Definition 5 (BSTMSIS $_{H,n,q,k,\ell,\gamma}$). Let $H : \{0, 1\}^* \times \mathcal{M} \rightarrow \mathcal{R}_2$ be a cryptographic hash function, where $M \subseteq \{0, 1\}^*$ is the message space. Let $q, k, \ell > 0$, $\gamma \geq 0$, and $\mathcal{R}$ of dimension n . In the (Q)ROM, the advantage of a polynomial-time adversary $\mathcal{A}_3$ that (quantumly) accesses the hash function H in solving the search BSTMSIS $_{H,n,q,k,\ell,\gamma}$ problem with respect to $\mathbf{j} \in \mathcal{R}_2^k \setminus \{0\}$ is

$$\text{Adv}_{H,n,q,k,\ell,\gamma}^{\text{BSTMSIS}}(\mathcal{A}_3) = \Pr \left[\begin{array}{c} 0 < \|\mathbf{y}\|_\infty < \gamma \wedge \\ H(\mathbf{A}\mathbf{y} - q\mathbf{c}\mathbf{j} \bmod 2q, \mu) = c \end{array} \mid \begin{array}{c} (\mathbf{A}_0|\mathbf{b}) \leftarrow \mathcal{R}_q^{k \times \ell}; \\ \mathbf{A} = (2\mathbf{b} + q\mathbf{j}|2\mathbf{A}_0|2\mathbf{I}_k) \bmod 2q; \\ (\mathbf{y}, c, \mu) \in \mathcal{R}_q^{k+\ell} \times \mathcal{R}_2 \times \mathcal{M} \leftarrow \mathcal{A}_3^{|H(\cdot)|}(\mathbf{A}) \end{array} \right].$$

In the classical ROM, the forking lemma can be used to reduce MSIS to SelfTargetMSIS, and the same reduction can be applied to the BSTMSIS problem. However, because this involves the forking lemma, these two reductions cannot be directly extended to QROM reductions. It should be noted that every coefficient of the challenge polynomial c used in the CS signature scheme belongs to $\{0, \pm 1\}$. For this reason, we slightly adjust the traditional proof and complete the reduction from MSIS to BSTMSIS in the classical ROM under the condition that every coefficient of the challenge polynomial c belongs to $\{0, \pm 1\}$.

Lemma 4.1 (Classical reduction from MSIS to BSTMSIS). *Let $q > 0$ be an odd modulus, $H : \{0,1\}^* \times \mathcal{M} \rightarrow \mathcal{R}_2$ a cryptographic hash function modelled as a random oracle, and assume that every polynomial-time classical algorithm has negligible advantage in $\text{MSIS}_{n,q,k,\ell,\gamma}$. Then every polynomial-time classical algorithm also has negligible advantage in $\text{BSTMSIS}_{n,q,k,\ell,\gamma/2}$.*

Proof. Consider a $\text{BSTMSIS}_{n,q,k,\ell,\gamma/2}$ classical algorithm $\mathcal{A}$, which runs in polynomial time and has classical access to H . If $\mathcal{A}^{H(\cdot)}(\mathbf{A})$ makes Q hash queries $H(\mathbf{w}_i, \mu_i)$ for $i = 1, \dots, Q$ and outputs a solution $(\mathbf{y}, c, \mu_j)$ for some $j \in [Q]$, then we can construct an adversary $\mathcal{A}'$ against $\text{MSIS}_{n,q,k,\ell,\gamma}$ as follows.

The adversary $\mathcal{A}'$ first rewinds $\mathcal{A}$ to the moment just before its j -th query and reprograms the hash function to $H(\mathbf{w}_j, \mu_j) = c' (\neq c)$. Then, with probability approximately $1/Q$, the algorithm $\mathcal{A}$ produces another solution $(\mathbf{y}', c', \mu_j)$. Thus we have

$$\begin{cases} \mathbf{A}\mathbf{y} - qc\mathbf{j} = \mathbf{w}_j = \mathbf{A}\mathbf{y}' - qc'\mathbf{j} \bmod 2q, \\ \|\mathbf{y}\|_\infty, \|\mathbf{y}'\|_\infty < \gamma/2. \end{cases}$$

Since q is odd, we have $\mathbf{A}(\mathbf{y} - \mathbf{y}') = (c - c')\mathbf{j} \bmod 2$. Because $c' \neq c$, and the Hamming weight of c in the CS scheme is τ with entropy at least $2^{-\lambda}$, the probability that $c' = c \bmod 2$ is $2^{-\lambda+\tau} \leq 2^{-\lambda/2}$, which is negligible; therefore the probability that $\mathbf{y} - \mathbf{y}' = 0$ over the integers is also negligible. Since $(\mathbf{b}|\mathbf{A}_0|\mathbf{I}_k) \cdot (\mathbf{y} - \mathbf{y}') = 0 \bmod q$ and $\|\mathbf{y} - \mathbf{y}'\|_\infty < \gamma$, it gives an $\text{MSIS}_{n,q,k,\ell,\gamma}$ solution for the matrix $(\mathbf{b}|\mathbf{A}_0|\mathbf{I}_k)$, where the submatrix $(-\mathbf{b}|\mathbf{A}_0) \in \mathcal{R}_q^{k \times \ell}$ is uniformly distributed. $\square$

Definition 4.3 (Standard Identification Protocol with Aborts (CID)). *A standard identification protocol with aborts ID is a three-round interactive protocol for a relation R between a prover P and a verifier V . The challenge is generated by $f(w)$ and follows a distribution $U(C)$ over a challenge set C . The prover holds a pair $(x, y) \in R$, while the verifier only knows x , where the pair (x, y) is generated by a PPT algorithm Gen , called the instance generator. The event “ $z = \perp$ ” is called an abort, and its probability β is called the abort probability.*

The standard identification schemes considered in this paper satisfy the following: given c and z , there exists a unique w such that V accepts, and w can be efficiently computed from c and z .

We need the following zero-knowledge variant, called perfectly accepting honest-verifier zero-knowledge.

Definition 4.4 (Perfectly accepting honest-verifier zero-knowledge). *Let ID be a standard identification scheme. It is said to satisfy paHVZK if there exists a PPT simulator Sim such that for any (x, y) generated by (1^λ) , the output distribution (w, c, z) of the interaction $\langle P(x, y) \leftrightarrow V(x) \rangle_{U(C)}$ conditioned on $z \neq \perp$ is identical to the distribution of (w', c', z') generated by $\text{Sim}(x)$.*

To avoid using the same commitment twice, we require the commitment to have high min-entropy.

Definition 4.5 (Commitment min-entropy). *For $\alpha \geq 0$, an identification scheme ID (with instance generator Gen) is said to have commitment min-entropy α if for all $(x, y) \leftarrow \text{Gen}(1^\lambda)$,*

$$H_\infty[w \mid (w, c, z) \leftarrow \langle P(x, y) \leftrightarrow V(x) \rangle_{U(C)}] \geq \alpha,$$

where H_∞ denotes min-entropy.

Finally, we need the notion of computational unique response in order to achieve strong unforgeability of the final signature scheme.

Definition 4.6 (Computational unique response). *Let ID be a standard identification scheme with instance generator Gen. The advantage $\text{Adv}_{\text{ID}}^{\text{CUR}}(\mathcal{A})$ of an adversary $\mathcal{A}$ against the CUR property of ID is the probability that it computes (w, c, z, z') such that $V(x)$ accepts both (w, c, z) and (w, c, z') with $z \neq z'$, where the input x is generated by $(x, y) \leftarrow \text{Gen}(1^\lambda)$.*

Definition 4.7 (Digital signature algorithm). *A digital signature algorithm consists of three probabilistic polynomial-time algorithms: the key generation algorithm KeyGen, the signing algorithm Sign, and the verification algorithm Verify, as follows:*

- **KeyGen** (1^λ): $1^\lambda \rightarrow (\text{vk}, \text{sk})$, i.e. on input the security parameter λ , it outputs the verification key vk and the signing key sk .
- **Sign** : $(\text{sk}, \mu) \rightarrow \sigma$, i.e. on input the signing key sk and a message μ , it outputs a signature σ .
- **Verify** : $(\text{vk}, \mu, \sigma) \rightarrow b \in \{0, 1\}$, i.e. on input the verification key vk , a message μ , and a signature σ , it outputs reject or accept, and is a deterministic algorithm.

Let $\gamma > 0$. If for every key pair (vk, sk) output by KeyGen and every message μ ,

$$\Pr[\text{Verify}(\text{vk}, \mu, \text{Sign}(\text{sk}, \mu)) = 1] \geq \gamma,$$

where the probability is over the randomness of the signing algorithm, the signature algorithm is said to be γ -correct. If the condition still holds when the probability is also over the randomness of the random oracle used by the scheme, the signature algorithm is said to be correct in the (Q)ROM.

The security notions for digital signature algorithms are existential unforgeability under no-message attack (UF-NMA), existential unforgeability under chosen-message attack (UF-CMA), and strong existential unforgeability under chosen-message attack (sUF-CMA).

Definition 4.8 (Unforgeability of signature schemes). *For a polynomial-time quantum adversary $\mathcal{A}$ that has access to the quantum random oracle, the advantage of $\mathcal{A}$ in breaking the UF-NMA security of the signature scheme is defined as*

$$\Pr_{(\text{vk}, \text{sk})}[\text{Verify}(\text{vk}, \mu^*, \sigma^*) = 1 \mid (\mu^*, \sigma^*) \leftarrow \mathcal{A}^H(\text{vk})] \leq \delta,$$

where the randomness is over $\mathcal{A}$ and $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$. If the adversary is allowed to access a signing oracle but cannot ask for a signature on the message μ^* , the above advantage is defined as the advantage of $\mathcal{A}$ in breaking the UF-CMA security of the signature scheme. Furthermore, if the adversary is allowed to ask for a signature on its message μ^* but its output is not a signature that has already been queried, the above advantage is defined as the advantage of $\mathcal{A}$ in breaking the sUF-CMA security of the signature scheme.

4.2 Signature Scheme Security

Like other Fiat-Shamir lattice-based signature schemes, we prove the theoretical security of the CS signature scheme in two steps. First, [Lemma 4.2](#) proves that the UF-NMA security of the CS signature scheme can be reduced to the hardness of the MLWE problem and the BSTMSIS problem; the latter can be reduced to the corresponding MSIS problem in the ROM via the forking lemma. Then, we construct a CS identification protocol that yields the CS signature scheme via the Fiat-Shamir transform, and use [Lemma 4.3](#) to prove that the CS identification protocol has the following properties: the commitment has min-entropy at least n , zero-knowledge, and computational unique response. From these properties, the (s)UF-CMA security of the CS signature scheme can be reduced to its UF-NMA security. Finally, combining the two steps, [Theorem 4.1](#) proves the theoretical security of the CS signature scheme. It should be noted that, by [Lemma B.3](#), the approximation of the rejection probability p_i in the CS signature scheme reduces the security by at most 1 bit; here we ignore this effect and assume that the rejection probability of the CS signature scheme is p_i .

Lemma 4.2 (UF-NMA security). *If there exists an adversary $\mathcal{A}$ that breaks the UF-NMA security of the CS signature scheme, then there exist two adversaries $\mathcal{B}$ and $\mathcal{C}$ with running time similar to that of $\mathcal{A}$ such that*

$$\text{Adv}_{CS}^{\text{UF-NMA}}(\mathcal{A}) \leq \text{Adv}_{n,q,k,\ell,\eta}^{\text{MLWE}}(\mathcal{B}) + \text{Adv}_{H,n,q,k,\ell+1,B'_2}^{\text{BSTMSIS}}(\mathcal{C}).$$

Proof. For convenience, we assume that the public key is $pk = (\mathbf{A}', \mathbf{b}) \in \mathcal{R}_q^{k \times \ell} \times \mathcal{R}_q^k$. Let pk' be chosen uniformly at random from $\mathcal{R}_q^{k \times \ell} \times \mathcal{R}_q^k$; we call the variant of the CS signature scheme with public key pk' the CSU signature scheme. Adversary $\mathcal{B}$ can invoke adversary $\mathcal{A}$ and distinguish pk from pk' according to the different behaviours of $\mathcal{A}$ against the CS signature scheme and the CSU signature scheme. Specifically, if $\mathcal{A}$ produces a valid forgery, the public key is deemed to be pk ; otherwise it is deemed to be pk' , i.e. solving the decisional $\text{MLWE}_{n,q,k,\ell,\eta}$ problem.

Adversary $\mathcal{C}$ constructs a solution to the corresponding BSTMSIS problem from a valid signature forgery output by $\mathcal{A}$. When $\mathcal{A}$ outputs a valid forgery $(\mu, \sigma = (\sigma_0, \sigma_1, \sigma_2, c))$, adversary $\mathcal{C}$ runs the signature verification [Algorithm 12](#) to obtain a polynomial vector $\mathbf{z}' = (\mathbf{z}'_0, \mathbf{z}'_1, \mathbf{z}'_2)$ with infinity norm less than B'_2 , a polynomial $\mathbf{w}'$, and a polynomial w' , satisfying $H(\mathbf{w}', w', \mu) = c$. Further, by [Theorem 3.1](#),

$$\alpha \mathbf{w}' + w' \mathbf{j} = \mathbf{A} \mathbf{z}' - q c \mathbf{j} \bmod 2q.$$

Let $H(\mathbf{w}', w', \mu) = H'(\alpha \mathbf{w}' + w' \mathbf{j}, \mu)$. Ignoring the difference between H and H' because of the randomness of the hash function, adversary $\mathcal{C}$ obtains a $\text{BSTMSIS}_{H,n,q,k,\ell+1,B'_2}$ solution $(\mathbf{z}', c, \mu)$. $\square$

The corresponding CS identification protocol is described as follows:

1. The prover generates $\mathbf{y}$ as in the signing algorithm [Algorithm 11](#), then computes $\mathbf{w} = \mathbf{A}\mathbf{y} \bmod 2q$, computes $\mathbf{w}^* = \text{HighBits}^h(\mathbf{w}, \alpha)$ and $w^* = \text{LSB}(\mathbf{y}_0)$, and sends them to the verifier.
2. The verifier randomly generates a challenge polynomial c and sends it to the prover.
3. Combining the challenge polynomial c , the prover generates a signature σ as in the signing algorithm [Algorithm 11](#) and sends it to the verifier.
4. The verifier runs the signature verification [Algorithm 12](#) and obtains $\mathbf{z}' = (\mathbf{z}'_0, \mathbf{z}'_1, \mathbf{z}'_2)$; if $\|\mathbf{z}'_0\|_\infty \geq B'_0$, $\|\mathbf{z}'_1\|_\infty \geq B'_1$, $\|\mathbf{z}'_2\|_\infty \geq B'_2$, and $\mathbf{A}\mathbf{z}' - qc\mathbf{j} = \alpha \mathbf{w}^* + w^* \mathbf{j} \bmod 2q$, then it accepts.

Lemma 4.3 (Properties of the CS identification protocol). *The CS identification protocol satisfies the following properties:*

1. The commitment $\mathbf{w}^*, w^*$ of the CS identification protocol has min-entropy at least n .
2. The CS identification protocol satisfies the paHVZK property.
3. If an adversary $\mathcal{A}$ can break the computational unique response property of the CS identification protocol, then there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ with similar running time that can solve the $\text{MLWE}_{n,q,k,\ell,\eta}$ and $\text{MSIS}_{n,q,k,\ell+1,2B'_2}$ problems, i.e.

$$\text{Adv}_{CS}^{CUR}(\mathcal{A}) \leq \text{Adv}_{n,q,k,\ell,\eta}^{MLWE}(\mathcal{B}) + \text{Adv}_{n,q,k,\ell+1,2B'_2}^{MSIS}(\mathcal{C}).$$

Proof. We first prove the min-entropy property. The commitment of the CS identification protocol is $\mathbf{w}^*, w^*$, where the entropy of $w^* = \text{LSB}(\mathbf{y}_0)$ is close to n ; adding the commitment $\mathbf{w}^*$, the min-entropy of the commitment is at least n .

We next prove the paHVZK property of the CS identification protocol. With only the public key, $\mathbf{b}_0$, and the ability to control the hash function, consider the following simulator:

1. Generate a polynomial vector $\mathbf{z} = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{z}_2)$, where $\mathbf{z}_0 \leftarrow \overline{\mathcal{D}}_{a_0,b_0}^{B_0-N}$, $\mathbf{z}_1 \leftarrow \overline{\mathcal{D}}_{a_1,b_1}^{B_1-\tau}$, and $\mathbf{z}_2 \leftarrow \overline{\mathcal{D}}_{a_2,b_2}^{B_2-\tau}$.
2. Randomly generate the challenge polynomial c , and compute the challenge polynomials c'_i in each iteration.
3. Randomly generate the bimodal random bits bit_i , and let $c_r = \sum_{i=0}^{N-1} (-1)^{\text{bit}_i} c'_i$.
4. Compute $\mathbf{w} = \mathbf{A}\mathbf{z} - qc\mathbf{j} - 2c_r \mathbf{b}_0 \bmod 2q$, and send the commitment $(\mathbf{w}^*, w^*) = (\text{HighBits}^h(\mathbf{w}, \alpha), \text{LSB}(\mathbf{z}_0 - c))$ to the verifier.

5. Let $c = H(\mathbf{w}^*, w^*, \mu)$.
6. Compute $\mathbf{h} = \mathbf{w}^* - \text{HighBits}^h(\mathbf{w} - 2\mathbf{z}_2 + 2c_r\mathbf{b}_0, \alpha) \bmod \pm \frac{2(q-1)}{\alpha}$.
7. Send the signature $\sigma = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{h}, c)$ to the verifier.

The $\mathbf{z}$ generated in the first step of the simulator has the same distribution as that generated by the signing algorithm [Algorithm 11](#). Since $\mathbf{z}$ is independent of bit_i , c , and $\mathbf{s}$, for any bit_i , c_r , and $\mathbf{s}$ there always exists a $\mathbf{y}$ such that $\mathbf{z} = \mathbf{y} + c_r\mathbf{s}$. Therefore, the value $\mathbf{w}$ generated in step 4 of the simulator satisfies

$$\begin{aligned} \mathbf{w} &= \mathbf{A}\mathbf{z} - qc\mathbf{j} - 2c_r\mathbf{b}_0 \bmod 2q \\ &= \mathbf{A}(\mathbf{y} + c_r\mathbf{s}) - qc\mathbf{j} - 2c_r\mathbf{b}_0 \bmod 2q \\ &= \mathbf{A}\mathbf{y} \bmod 2q. \end{aligned}$$

Moreover, $LSB(\mathbf{z}_0 - c) = LSB(\mathbf{y}_0 + c_r - c) = LSB(\mathbf{y}_0)$. Therefore, the above simulator perfectly simulates $\mathbf{A}\mathbf{y}$ and $LSB(\mathbf{y}_0)$ of the signing algorithm, respectively, and thus perfectly simulates a (non-abort) verifier. Consequently, the CS identification protocol satisfies the paHVZK property.

We now prove that the CS identification protocol has the computational unique response property. If the CS identification protocol does not have computational unique response, then adversary $\mathcal{A}$ can output two valid transcripts $((\mathbf{w}^*, w^*), c, \sigma = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{h}, c))$ and $((\mathbf{w}^*, w^*), c, \sigma' = (\mathbf{z}'_0, \mathbf{z}'_1, \mathbf{h}', c))$ of the identification protocol. By the verification process of the identification protocol, adversary $\mathcal{C}$ knows that $\mathbf{A}\mathbf{z} - qc\mathbf{j} = \mathbf{A}\mathbf{z}' - qc\mathbf{j} \bmod 2q$, i.e. $\mathbf{A}(\mathbf{z} - \mathbf{z}') = 0 \bmod 2q$, i.e.

$$(-\beta\mathbf{b}_1, \mathbf{A}', \mathbf{I}_k)(\mathbf{z} - \mathbf{z}') = 0 \bmod q.$$

If $(\mathbf{z}_0, \mathbf{z}_1) = (\mathbf{z}'_0, \mathbf{z}'_1)$, then $\mathbf{h}' \neq \mathbf{h}$, so $\mathbf{z}_2 \neq \mathbf{z}'_2$ and $\mathbf{z} - \mathbf{z}' \neq 0$; otherwise $(\mathbf{z}_0, \mathbf{z}_1) \neq (\mathbf{z}'_0, \mathbf{z}'_1)$, so $\mathbf{z} \neq \mathbf{z}'$, and thus adversary $\mathcal{C}$ has found a solution to $(-\beta\mathbf{b}_1, \mathbf{A}', \mathbf{I}_k)\mathbf{x} = 0 \bmod q$ with $\|\mathbf{x}\|_\infty \leq 2B'_2$.

However, $(-\beta\mathbf{b}_1, \mathbf{A}')$ can be weakened to an $\text{LWE}_{n,q,k,\ell,\eta}$ instance $(-\mathbf{b}, \mathbf{A}')$. Let the CUS identification protocol be the CS identification protocol in which $(-\mathbf{b}, \mathbf{A}')$ is replaced by a random instance from $\mathcal{R}_q^k \times \mathcal{R}_q^{k \times \ell}$. Then adversary $\mathcal{B}$ gives either the CUS or the CS identification protocol to adversary $\mathcal{A}$; if $\mathcal{A}$ outputs two valid transcripts $((\mathbf{w}^*, s^*), c, \sigma = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{h}, c))$, then adversary $\mathcal{B}$ judges that the input identification protocol is based on the LWE instance $(-\mathbf{b}, \mathbf{A}')$; otherwise it is deemed to be based on a random element of $\mathcal{R}_q^k \times \mathcal{R}_q^{k \times \ell}$, thereby solving the decisional $\text{MLWE}_{n,q,k,\ell,\eta}$ problem. $\square$

Combining [Lemma 4.2](#) and [Lemma 4.3](#), [Theorem 4.1](#) proves that the sUF-CMA security of the CS signature scheme can be reduced to the hardness of the MLWE, MSIS, and BSTMSIS problems.

Theorem 4.1 (Theoretical security of the CS signature scheme). *Assume that the min-entropy of the CS identification protocol is β_1 and the probability of an empty output is β_2 . Assume that adversary $\mathcal{A}$ can make q_s signing queries and q_h hash queries. Then there exist adversaries $\mathcal{B}, \mathcal{C}$ such that*

$$\begin{aligned}
Adv_{CS}^{UF-CMA}(\mathcal{A}) &\leq Adv_{n,q,k,\ell,\eta}^{MLWE}(\mathcal{B}) + Adv_{H,n,q,k,\ell+1,B'_2}^{BSTMSIS}(\mathcal{C}) \\
&\quad + \frac{2^{-\beta_1/2+1}q_s}{1-\beta_2} \sqrt{q_h + 1 + \frac{q_s}{1-\beta_2}} + 2^{-\beta_1/2+1}(q_h + 1) \sqrt{\frac{q_s}{1-\beta_2}}.
\end{aligned}$$

Furthermore, if $\mathcal{A}$ attacks the sUF -CMA security of the CS signature scheme, a similar conclusion holds:

$$\begin{aligned}
Adv_{CS}^{sUF-CMA}(\mathcal{A}) &\leq 2Adv_{n,q,k,\ell,\eta}^{MLWE}(\mathcal{B}) + Adv_{H,n,q,k,\ell+1,B'_2}^{BSTMSIS}(\mathcal{C}) + Adv_{n,q,k,\ell+1,2B'_2}^{MSIS}(\mathcal{D}) \\
&\quad + \frac{2^{-\beta_1/2+1}q_s}{1-\beta_2} \sqrt{q_h + 1 + \frac{q_s}{1-\beta_2}} + 2^{-\beta_1/2+1}(q_h + 1) \sqrt{\frac{q_s}{1-\beta_2}}.
\end{aligned}$$

4.3 Concrete Security Strength

The unforgeability of the CS signature scheme ([Theorem 4.1](#)) is ultimately reduced to the three hard problems MLWE, MSIS, and BSTMSIS. For the MLWE problem, we mainly consider the two most effective attacks: the primal attack [[BG14](#), [AGV17](#)] and the dual attack [[MR09](#), [ADPS16](#), [Alb17](#)]. For the MSIS and BSTMSIS problems, we mainly consider the infinity-norm attack; more specifically, like HAETAE [[CHE⁺24](#)], ML-DSA [[LDK⁺22](#)], and [[Gar25](#)], we treat the $BSTMSIS_{H,n,q,k,\ell+1,B'_2}$ problem as the $MSIS_{H,n,q,k,\ell+1,B'_2}$ problem. In addition, in the concrete parameter evaluation, we view an $MLWE_{n,q,k,\ell,\eta}$ instance as an ordinary LWE instance of dimension $nk \times n\ell$, and an $MSIS_{n,q,k,\ell,\gamma}$ instance as an ordinary SIS instance.

BKZ and core-SVP evaluation framework. The best known attacks on LWE/SIS are based on blockwise BKZ lattice-basis reduction [[SE94](#), [CN11](#), [HPS11](#)]. The BKZ algorithm is used to solve the Shortest Vector Problem (SVP) in a d -dimensional lattice by iteratively running sieving (or enumeration) to solve smaller b -dimensional projected sublattice SVP instances in order to find a short vector in the d -dimensional lattice. Its time complexity is dominated by the sieving step. In the Core-SVP model, the BKZ algorithm is assumed to run sieving only once. Under classical attacks, sieving [[BDGL⁺16](#), [CL21](#)] costs approximately $2^{0.292b}$; under quantum attacks, sieving costs approximately $2^{0.257b}$.

Primal attack. The primal attack is used to solve the search LWE problem, i.e. to recover the secret vector $\mathbf{s}$ from the LWE samples. It converts the LWE key-recovery problem into a short-vector problem in a q -ary lattice. Given LWE samples $(\mathbf{A}, \mathbf{b} = \mathbf{As} + \mathbf{e}) \in \mathbb{Z}_q^{m \times l} \times \mathbb{Z}_q^m$, construct the lattice

$$\Lambda = \{(\mathbf{s}', \mathbf{e}', 1) \in \mathbb{Z}^{l+m+1} : \mathbf{As}' + \mathbf{e}' - \mathbf{b} = \mathbf{0} \bmod q\}.$$

Because there exists a short vector $(\mathbf{s}, \mathbf{e})$ satisfying $\mathbf{b} = \mathbf{As} + \mathbf{e} \bmod q$, the vector $(\mathbf{s}, \mathbf{e}, 1)$ is a short vector in Λ . The primal attack runs the BKZ algorithm on Λ ; if a lattice vector shorter than $(\mathbf{s}, \mathbf{e}, 1)$ is found, the search LWE problem is considered solved. In concrete evaluations, the minimal attack cost is found by searching over the number of LWE samples m and the BKZ block size b .

Dual attack. The dual attack is used to solve the decisional LWE problem. Given LWE samples $(\mathbf{A}, \mathbf{b})$, define the dual lattice

$$\Lambda' = \{(\mathbf{u}, \mathbf{v}) \in \mathbb{Z}^m \times \mathbb{Z}^l : \mathbf{A}^\top \mathbf{u} + \mathbf{v} = \mathbf{0} \bmod q\}.$$

If $(\mathbf{u}, \mathbf{v})$ is a short vector of Λ' , then

$$\mathbf{u}^\top \mathbf{b} = -\mathbf{v}^\top \mathbf{s} + \mathbf{u}^\top \mathbf{e}.$$

When $\mathbf{b}$ is indeed an LWE instance, the right-hand side is the sum of two small inner products, so $\mathbf{u}^\top \mathbf{b}$ takes a small value modulo q ; when $\mathbf{b}$ is uniformly random, $\mathbf{u}^\top \mathbf{b}$ is uniformly distributed over $\mathbb{Z}_q$. After collecting enough short dual vectors, the attacker can distinguish an LWE instance from a uniform instance by a statistical test, thereby solving the decisional LWE problem. In concrete evaluations, the minimal attack cost is found by searching over the number of LWE samples m and the BKZ block size b .

Infinity-norm attack. For the SIS problem in the infinity norm, a currently effective attack is the infinity-norm attack [LDK⁺22]. Given $\mathbf{A} \in \mathbb{Z}_q^{m \times l}$ and a norm bound γ , construct the q -ary kernel lattice

$$\Lambda'' = \{\mathbf{u} \in \mathbb{Z}^{m+l} : \mathbf{B}\mathbf{u} = \mathbf{0} \bmod q\},$$

where $\mathbf{B} = (\mathbf{A} | \mathbf{I}_m)$. Running the BKZ algorithm on Λ'' to find a short non-zero vector gives a solution to the SIS problem. In concrete evaluations, the minimal attack cost is found by searching over l and the BKZ block size b .

In Table 4 we give the concrete security strength of $\text{MLWE}_{n,q,k,\ell,\eta}$ under the primal and dual attacks, and the security strength of $\text{MSIS}_{n,q,k,\ell+1,B'_2}$ and $\text{MSIS}_{n,q,k,\ell+1,2B'_2}$ under the infinity-norm attack at each security level.

4.4 Other Security Features

One design goal of the CS scheme is to be free of floating-point operations throughout the whole process. The mask vector $\mathbf{y}$ is sampled from the Centered Multinomial Distribution $\mathcal{D}_{a,b}$, which only involves uniform sampling of short integers and addition/subtraction; in the rejection sampling, acceptance is performed with probability close to p_i using only integer operations. These designs make the CS scheme easy to implement in constant time, thereby improving its resistance to side-channel attacks.

5 Performance and Features

5.1 Performance

To evaluate the practical performance of the CS scheme on general-purpose software platforms, we implemented the three parameter sets CS-128, CS-256, and CS-512 in standard C and

Table 4: Concrete security strength of the CS signature scheme

λ	Security parameter	128	256	512
Primal attack	BKZ block size	450	978	1777
	Classical security strength	131.4	285.58	518.88
	Quantum security strength	115.33	250.66	455.45
Dual attack	BKZ block size	445	971	1766
	Classical security strength	119.94	283.5	515.67
	Quantum security strength	114.05	248.87	452.63
Infinity norm	BKZ block size	441(374)	916(797)	1879(1660)
	Classical security strength	128.81 (109.21)	267.47 (232.92)	548.67(484.72)
	Quantum security strength	113.08 (95.86)	234.77 (204.47)	481.59(425.46)

measured their operation efficiency and data sizes in a unified test environment. The detailed configuration of the test platform is shown in [Table 5](#).

Table 5: Software implementation platform configuration

Item	Configuration
CPU	Intel Core i3-6100 @ 3.70GHz
Memory	16GB
Operating system	Windows 7 64-bit
Programming language	Standard C
Compiler	Microsoft Visual Studio 2022 (MSVC)
Instruction-set optimisation	None
Cryptographic-library dependencies	None

This implementation does not rely on any dedicated cryptographic library, nor does it use SIMD/AVX or other dedicated instruction-set acceleration, or assembly-level optimisation for a specific processor; therefore it can serve as a reference-implementation performance of the CS scheme on general-purpose x86-64 platforms.

Operation efficiency. [Table 2](#) gives the efficiency comparison of key generation, signing, and verification among the CS scheme, HAETAE, and ML-DSA on the same test platform (unit: kcycles, i.e. thousands of cycles). At the 128 security level, CS-128 requires approximately 503, 4009, and 536 kcycles for key generation, signing, and verification, respectively; at the 256 security level, CS-256 requires approximately 809, 13083, and 1028 kcycles for the corresponding operations; at the 512 security level, CS-512 requires approximately 2466, 45269, and 3127 key-

cles. The experimental results show that the CS scheme has a clear advantage in key-generation efficiency and competitive verification efficiency. The signing process is relatively time-consuming because it involves iterative bimodal rejection sampling and integer-only approximation of the rejection probability, but it is still faster than the signing generation of HAETAE at the same security level.

Transmission and storage overhead. Table 1 gives the public-key, signature, and secret-key size comparison among the CS scheme, HAETAE, and ML-DSA (unit: bytes). The public-key, signature, and secret-key sizes of CS-128 are 976 B, 1548 B, and 1888 B, respectively; those of CS-256 are 1760 B, 3164 B, and 3968 B; those of CS-512 are 4288 B, 5975 B, and 7808 B. The CS scheme has a clear advantage in public-key size: at the 128 and 256 security levels its public key is the smallest of the three schemes. Its signature size is also compact, and the sum of public-key and signature sizes is close to the minimum.

5.2 Features

Innovation. The core innovations of the CS scheme can be summarised in four points: (1) it uses the bimodal Centered Multinomial Distribution as the mask distribution, whose sampling process involves no floating-point operations; it designs an integer-only method to approximate the rejection-sampling probability, and controls the approximation error by Rényi divergence, losing at most 1 bit of security; these two designs make the whole CS scheme free of floating-point operations. (2) It introduces the flexible iterative bimodal parameter τ' , achieving a better balance between sampling complexity and rejection-probability computation complexity. (3) The CS scheme is compatible with efficient public-key compression mechanisms, making the public key more compact. (4) It extends the coefficient space of the challenge polynomial in traditional bimodal FS signatures from $\{0, 1\}$ to $\{0, \pm 1\}$, reducing the Hamming weight for the same challenge entropy and thus reducing the signature size.

Simplicity. The CS scheme retains the classic three-step structure of Fiat-Shamir with Aborts signatures: commitment $\mathbf{w} = \mathbf{A}\mathbf{y}$, challenge c , and response $\mathbf{z}$. Its mask sampling only requires uniform sampling of short integers and addition/subtraction; the core polynomial multiplication is implemented by the NTT over $\mathcal{R}_q$, and the modulus q satisfies $q \equiv 1 \pmod{2n}$, so standard NTT libraries can be used directly. The rejection-probability computation mainly uses integer multiplication and right-shift operations, without floating-point operations. Overall, the CS scheme completes the whole process without floating-point operations through simple integer operations, significantly reducing implementation complexity and side-channel protection cost.

Flexibility. The CS scheme can be flexibly adjusted according to different application scenarios. First, the ring dimension n and the matrix dimensions k, ℓ of the underlying hard problem can be adjusted according to the target security level. Second, the mask vector $\mathbf{y}$ is divided into three parts, each of which can use a different centered multinomial distribution, and the Centered

Multinomial Distribution has two parameters a_i, b_i , giving it more flexibility than a Gaussian distribution. Third, the parameter τ' makes the iterative bimodal design more flexible: a smaller τ' reduces the repetition count of each rejection sampling but increases the number of iterations, while a larger τ' has the opposite effect. Fourth, the public-key compression parameter β and the signature compression parameter α can trade size against security. This flexible parameter selection allows CS to adapt to application scenarios with different requirements (higher security, smaller size, faster speed, etc.) by optimising the parameter configuration.

Scalability. CS provides three parameter sets, CS-128, CS-256, and CS-512, corresponding to approximately 128, 256, and 512 bits of classical core-SVP security strength, respectively (see Table 4). Because the CS scheme is highly parameter-flexible, it can adapt to different security levels by parameter adjustment.

References

- [ADPS16] Erdem Alkim et al. “Post-quantum Key Exchange — A New Hope”. In: *Proceedings of the 25th USENIX Security Symposium (USENIX Security 2016)*. USENIX Association, 2016, pp. 327–343. URL: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/alkim>.
- [AGV17] Martin R. Albrecht et al. “Revisiting the Expected Cost of Solving uSVP and Applications to LWE”. In: *Advances in Cryptology – ASIACRYPT 2017*. Vol. 10624. LNCS. Springer, 2017, pp. 297–322.
- [Alb17] Martin R. Albrecht. “On Dual Lattice Attacks Against Small-Secret LWE and Parameter Choices in HELib and SEAL”. In: *Advances in Cryptology – EUROCRYPT 2017*. Vol. 10211. LNCS. Springer, 2017, pp. 103–129. DOI: [10.1007/978-3-319-56614-6_4](https://doi.org/10.1007/978-3-319-56614-6_4).
- [BDGL⁺16] Anja Becker et al. “New Directions in Nearest Neighbor Searching with Applications to Lattice Sieving”. In: *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2016)*. SIAM, 2016, pp. 10–24. DOI: [10.1137/1.9781611974331.ch2](https://doi.org/10.1137/1.9781611974331.ch2).
- [BG14] Shi Bai and Steven D. Galbraith. “An Improved Compression Technique for Signatures Based on Learning with Errors”. In: *Topics in Cryptology – CT-RSA 2014*. Vol. 8366. LNCS. Springer, 2014, pp. 28–47. DOI: [10.1007/978-3-319-04852-9_2](https://doi.org/10.1007/978-3-319-04852-9_2).
- [CHE⁺23] Jung Hee Cheon et al. *HAETAE: Algorithm Specifications and Supporting Documentation*. Tech. rep. Korea Internet & Security Agency, May 2023. URL: <https://www.haetae.org/>.
- [CHE⁺24] Jung Hee Cheon et al. “HAETAE: Shorter Lattice-Based Fiat-Shamir Signatures”. In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2024.3 (2024), pp. 25–75. DOI: [10.46586/tches.v2024.i3.25-75](https://doi.org/10.46586/tches.v2024.i3.25-75).

- [CL21] André Chailloux and Johanna Loyer. “Lattice Sieving via Quantum Random Walks”. In: *Advances in Cryptology – ASIACRYPT 2021, Part IV*. Vol. 13093. LNCS. Springer, 2021, pp. 63–91. DOI: [10.1007/978-3-030-92068-5_3](https://doi.org/10.1007/978-3-030-92068-5_3).
- [CN11] Yuanmi Chen and Phong Q. Nguyen. “BKZ 2.0: Better Lattice Security Estimates”. In: *Advances in Cryptology – ASIACRYPT 2011*. Vol. 7073. LNCS. Springer, 2011, pp. 1–20. DOI: [10.1007/978-3-642-25385-0_1](https://doi.org/10.1007/978-3-642-25385-0_1).
- [DDLL13] Léo Ducas et al. “Lattice Signatures and Bimodal Gaussians”. In: *Advances in Cryptology – CRYPTO 2013, Part I*. Vol. 8042. LNCS. Springer, 2013, pp. 40–56. DOI: [10.1007/978-3-642-40041-4_3](https://doi.org/10.1007/978-3-642-40041-4_3).
- [Dud09] Jarek Duda. *Asymmetric Numeral Systems*. 2009. arXiv: [0902.0271](https://arxiv.org/abs/0902.0271). URL: <https://arxiv.org/abs/0902.0271>.
- [FIPS204] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204. National Institute of Standards and Technology, Aug. 2024. DOI: [10.6028/NIST.FIPS.204](https://doi.org/10.6028/NIST.FIPS.204). URL: <https://csrc.nist.gov/pubs/fips/204/final>.
- [Gar25] Joel Gärtner. “Compact Lattice Signatures via Iterative Rejection Sampling”. In: *Advances in Cryptology – CRYPTO 2025*. Vol. 16000. LNCS. Springer, 2025, pp. 514–547. DOI: [10.1007/978-3-032-01855-7_17](https://doi.org/10.1007/978-3-032-01855-7_17).
- [Gie14] Fabian Giesen. *Interleaved Entropy Coders*. 2014. arXiv: [1402.3392](https://arxiv.org/abs/1402.3392). URL: <https://arxiv.org/abs/1402.3392>.
- [HPS11] Guillaume Hanrot, Xavier Pujol, and Damien Stehlé. “Analyzing Blockwise Lattice Algorithms Using Dynamical Systems”. In: *Advances in Cryptology – CRYPTO 2011*. Vol. 6841. LNCS. Springer, 2011, pp. 447–464. DOI: [10.1007/978-3-642-22792-9_25](https://doi.org/10.1007/978-3-642-22792-9_25).
- [LDK⁺22] Vadim Lyubashevsky et al. *CRYSTALS-DILITHIUM*. Tech. rep. National Institute of Standards and Technology, 2022. URL: <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [Lyu09] Vadim Lyubashevsky. “Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures”. In: *Advances in Cryptology – ASIACRYPT 2009*. Vol. 5912. LNCS. Springer, 2009, pp. 598–616. DOI: [10.1007/978-3-642-10366-7_35](https://doi.org/10.1007/978-3-642-10366-7_35).
- [Lyu12] Vadim Lyubashevsky. “Lattice Signatures without Trapdoors”. In: *Advances in Cryptology – EUROCRYPT 2012*. Vol. 7237. LNCS. Springer, 2012, pp. 738–755. DOI: [10.1007/978-3-642-29011-4_43](https://doi.org/10.1007/978-3-642-29011-4_43).
- [MR09] Daniele Micciancio and Oded Regev. “Lattice-based Cryptography”. In: *Post-quantum Cryptography*. Springer, 2009, pp. 147–191.

- [Pre17] Thomas Prest. “Sharper Bounds in Lattice-Based Cryptography Using the Rényi Divergence”. In: *Advances in Cryptology – ASIACRYPT 2017*. Vol. 10624. LNCS. Springer, 2017, pp. 347–374. DOI: [10.1007/978-3-319-70694-8_12](https://doi.org/10.1007/978-3-319-70694-8_12).
- [Sch91] Claus-Peter Schnorr. “Efficient Signature Generation by Smart Cards”. In: vol. 4. 3. Springer, 1991, pp. 161–174. DOI: [10.1007/BF00196725](https://doi.org/10.1007/BF00196725).
- [SE94] Claus-Peter Schnorr and Martin Euchner. “Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems”. In: *Mathematical Programming* 66.1-3 (1994), pp. 181–199. DOI: [10.1007/BF01581144](https://doi.org/10.1007/BF01581144).
- [WKH⁺25] Joo Woo et al. *NTRU+Sign: Compact NTRU-Based Signatures Using Bimodal Distributions*. Cryptology ePrint Archive, Report 2025/106. 2025. URL: <https://eprint.iacr.org/2025/106>.

Appendix A Properties of the centered multinomial distribution

This appendix introduces some properties of the centered multinomial distribution. The probability that $x \sim \mathcal{D}_{a,b}$ is $\frac{f(x)}{2^{2ab}}$, where

$$f_{a,b}(x) = \sum_{i=0}^{\lfloor \frac{x+a(2^b-1)}{2^b} \rfloor} (-1)^i \binom{2a}{i} \binom{x+a(2^b+1)-2^b i-1}{2a-1},$$

and $f_{a,b}(x)$ is abbreviated as $f(x)$ when no confusion arises. Accordingly, for a d -dimensional independent vector $\mathbf{x} \sim \mathcal{D}_{a,b}$, its probability is $\frac{g(\mathbf{x})}{2^{2abd}}$, where $g(\mathbf{x}) = \prod_{i=0}^{d-1} f(\mathbf{x}_i)$. Thus, the probabilities of $x \sim \mathcal{D}_{a,b,r}$ and $\mathbf{x} \sim \mathcal{D}_{a,b,\mathbf{r}}$ are

$$\frac{f(x-r)}{2^{2ab}}, \quad \frac{g(\mathbf{x}-\mathbf{r})}{2^{2abd}}.$$

The probabilities of $x \sim \overline{\mathcal{D}}_{a,b,r}^t$ and $\mathbf{x} \sim \overline{\mathcal{D}}_{a,b,\mathbf{r}}^t$ are

$$\frac{f(x-r)}{\sum_{y=-t}^{y=t} f(y-r)}, \quad \frac{g(\mathbf{x}-\mathbf{r})}{\prod_{i=0}^{d-1} \left(\sum_{\mathbf{y}_i=-t}^{\mathbf{y}_i=t} f(\mathbf{y}_i-\mathbf{r}_i) \right)}.$$

By the symmetry of the Centered Multinomial Distribution $\mathcal{D}_{a,b}$, the probability of $x \sim \overline{\mathcal{D}}_{a,b,\pm r}^t$ is

$$\frac{f(x-r) + f(x+r)}{\sum_{y=-t}^{y=t} f(y-r) + \sum_{y=-t}^{y=t} f(y+r)} = \frac{f(x-r) + f(x+r)}{2 \sum_{y=-t}^{y=t} f(y+r)}.$$

Similarly, by the symmetry of the Centered Multinomial Distribution $\mathcal{D}_{a,b}$, the probability of $\mathbf{x} \sim \overline{\mathcal{D}}_{a,b,\pm \mathbf{r}}^t$ is

$$\frac{g(\mathbf{x}-\mathbf{r}) + g(\mathbf{x}+\mathbf{r})}{\prod_{i=0}^{d-1} \left(\sum_{\mathbf{y}_i=-t}^{\mathbf{y}_i=t} f(\mathbf{y}_i-\mathbf{r}_i) \right) + \prod_{i=0}^{d-1} \left(\sum_{\mathbf{y}_i=-t}^{\mathbf{y}_i=t} f(\mathbf{y}_i+\mathbf{r}_i) \right)} = \frac{g(\mathbf{x}-\mathbf{r}) + g(\mathbf{x}+\mathbf{r})}{2 \prod_{i=0}^{d-1} \left(\sum_{\mathbf{y}_i=-t}^{\mathbf{y}_i=t} f(\mathbf{y}_i+\mathbf{r}_i) \right)}.$$

Lemma A.1. *The probability mass function $f(x)$ of the one-dimensional Centered Multinomial Distribution is log-concave, and for any integer a , $\frac{f^2(x)}{f(x+a)f(x-a)}$ is a monotone increasing function of $|a|$.*

Proof. The uniform distribution is log-concave, and $f(x)$ is a convolution of uniform distributions, so $f(x)$ is log-concave. Let $i > 0$. By the property of log-concave functions,

$$\frac{f(x)}{f(x+1)} \geq \frac{f(x-1)}{f(x)} \Leftrightarrow \frac{f(x+i)}{f(x+i+1)} \geq \frac{f(x-i-1)}{f(x-i)} \Leftrightarrow \frac{f(x-i)f(x+i)}{f(x-i-1)f(x+i+1)} \geq 1.$$

Let $h(a) = \frac{f^2(x)}{f(x+a)f(x-a)}$; then

$$h(|a|+1) = h(|a|) \frac{f(x+|a|)f(x-|a|)}{f(x-|a|-1)f(x+|a|+1)} \geq h(|a|).$$

□

Appendix B Lemmas for the Rejection Sampling Algorithm

Lemma B.1. *For positive integers b, c , let the relative errors of b', c' with respect to b, c be $\Delta_b = \frac{b'-b}{b}$ and $\Delta_c = \frac{c'-c}{c}$. Then the relative error Δ_{b+c} of $b' + c'$ with respect to $b + c$, the relative error Δ_{bc} of $b'c'$ with respect to bc , and the relative error $\Delta_{b/c}$ of b'/c' with respect to b/c satisfy*

$$\Delta_{b+c} = \frac{b\Delta_b + c\Delta_c}{b+c}, \quad \Delta_{bc} = \Delta_b + \Delta_c + \Delta_b\Delta_c, \quad \Delta_{b/c} = \frac{\Delta_b - \Delta_c}{1 + \Delta_c}.$$

Proof. The relative error of $b' + c'$ with respect to $b + c$ is

$$\Delta_{b+c} = \frac{b' + c'}{b + c} - 1 = \frac{b(1 + \Delta_b) + c(1 + \Delta_c)}{b + c} - 1 = \frac{b\Delta_b + c\Delta_c}{b + c}.$$

The relative error of $b'c'$ with respect to bc is

$$\Delta_{bc} = \frac{b'c'}{bc} - 1 = \frac{b(1 + \Delta_b)c(1 + \Delta_c)}{bc} - 1 = \Delta_b + \Delta_c + \Delta_b\Delta_c.$$

The relative error of b'/c' with respect to b/c is

$$\Delta_{b/c} = \frac{b'/c'}{b/c} - 1 = \frac{\Delta_b - \Delta_c}{1 + \Delta_c} = \frac{\Delta_b - \Delta_c}{1 + \Delta_c}.$$

□

Lemma B.2. *Let $d = n(k + \ell + 1)$. The relative error Δ between the probability $\frac{N}{D}$ of False output by [Algorithm 4](#) and p_i satisfies $\Delta \leq 2^{-L+4+\log d}$.*

Proof. [Algorithm 4](#) takes as input two d -dimensional vectors $\mathbf{z}, \mathbf{r}$. The algorithm approximates p_i using only integer operations; its error mainly comes from the right-shifts in lines 7 and 13 of [Algorithm 5](#). We first compute the relative error of a single right-shift in [Algorithm 5](#). For an

integer N with bit length greater than L , after a right-shift by s bits we obtain an integer N_h of length L bits satisfying

$$N = 2^s N_h + N_l$$

where

$$N_l \in [0, 2^s).$$

The relative error of a single right-shift is

$$\Delta' = \frac{N_l}{N} \leq \frac{2^s}{2^{s+L-1}} = 2^{1-L}.$$

The number of right-shifts involved in [Algorithm 5](#) is at most $2d$; by [Lemma B.1](#) (here $L = 64$ and Δ' is very small, so we simply assume $\Delta_{bc} = \Delta_b + \Delta_c$), the relative error Δ_0 of [Algorithm 5](#) satisfies $\Delta_0 \leq 2d\Delta' = 2^{-L+2+\log d}$.

Let the relative errors of the five calls to [Algorithm 5](#) in [Algorithm 4](#) be $\Delta_1, \Delta_2, \Delta_3, \Delta_4, \Delta_5$. Then

$$|\Delta_1| \leq 2^{-L+3+\log d}, \quad |\Delta_2| \leq 2^{-L+2+\log d}, \quad |\Delta_3| \leq 2^{-L+2+\log d}, \quad |\Delta_4| \leq 2^{-L+2+\log d}.$$

By [Lemma B.1](#), the relative error Δ_5 of the fifth call to [Algorithm 5](#) satisfies (here each Δ_i is small, so we simply assume $\Delta_{bc} = \Delta_b + \Delta_c$ and ignore the effect of one truncation relative error Δ' , as well as the 2^{1-L} error introduced by Approx)

$$|\Delta_5| \leq |\Delta_2| + \max(|\Delta_3|, |\Delta_4|).$$

Therefore, by [Lemma B.1](#), the relative error Δ of the final value $\frac{N}{D}$ with respect to p satisfies (here each Δ_i is small, so we simply assume $\Delta_{b/c} = \Delta_b - \Delta_c$)

$$|\Delta| \leq |\Delta_1| + |\Delta_5| \leq |\Delta_1| + |\Delta_2| + \max(|\Delta_3|, |\Delta_4|) \leq 2^{-L+4+\log d}.$$

□

Lemma B.3. *Let the CSE scheme be the CS scheme in which the probability of False output by the Reject algorithm is changed from $\frac{N}{D}$ to p_i . If the CSE scheme has $\lambda + 1$ bits of security under $q_s = 2^{64}$ signing queries, then the CS scheme has at least λ bits of security.*

Proof. By [Lemma B.2](#), the relative error Δ between p_i and $\frac{N}{D}$ satisfies $|\Delta| \leq 2^{-L+4+\log d}$, i.e. the relative error Δ_A between the probability $\prod_{i=1}^N p_{i-1}$ that the CSE signing algorithm outputs $\mathbf{z}_N$ and the probability that the CS signing algorithm outputs $\mathbf{z}_N$ satisfies $|\Delta_A| \leq N|\Delta|$ (here each Δ is small, so we simply assume $\Delta_{bc} = \Delta_b + \Delta_c$). In the CS scheme, $N \leq 20$, $d \leq 6144$, and $L = 64$; therefore $|\Delta_A| \leq 2^{-43}$. By the result in Section 3.3 of [\[Pre17\]](#), for $q_s = 2^{64}$ signing queries and $\lambda \leq 512$, if $|\Delta_A| \leq 2^{-38}$, then the CS scheme loses at most 1 bit of concrete security strength compared with the CSE scheme. □

Lemma B.4. (Similar to Lemma 2.1 of [\[D DLL13\]](#))

Using the definitions in Section 2.7, let $g'_i(\mathbf{x})$ and $g_i(\mathbf{x})$ be the probability mass functions of the integer distributions $\mathcal{D}'_i$ and $\mathcal{D}_i$, respectively. If there exists a constant M_i such that $p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})} \leq 1$, then the following two methods produce the same output distribution:

1. *Sample $\mathbf{x} \leftarrow \mathcal{D}'_i$, and output $\mathbf{x}$ with probability p_i .*
2. *Sample $\mathbf{x} \leftarrow \mathcal{D}_i$, and output $\mathbf{x}$ with probability $\frac{1}{M_i}$.*